DEPI PROJECT

NUVANTA

AI-Powered Enterprise Financial & Operations Management System

System Design Document

Analysis, Design & Data Modeling Deliverable

Document	System Design Document (DEPI deliverable)
Project	NUVANTA
Instructor	Ahmed Mohamed Maher
Track	Full-Stack .Net Web
Team	Menna Atef, Nour Hany, Weam Emad, Nourhan Fawzy, Omar Mohamed

CONTENTS

1. Problem Statement & Objectives
2. Database Design & Data Modeling
3. Data Flow & System Behavior
4. UI/UX Design & Prototyping
5. System Deployment & Integration
6. Additional Deliverables

1. Problem Statement & Objectives

1.1 Overview

NUVANTA is a single web platform that connects a company's money with its operations. The company sets a total budget, distributes it across departments and cost centers, and every purchase, expense, salary or sale is reflected automatically in the budget, the accounting ledger, the inventory and the analytics. Each user sees and does only what their role allows, and an AI layer helps management make better decisions.

1.2 Problem Statement

In many companies, budgets, purchasing, payments, inventory and accounting live in separate tools or spreadsheets. This causes:

- Budgets that are exceeded without anyone noticing in time.
- Purchases approved without checking available budget or cash.
- No clear difference between planned money, committed money, spent money and cash actually in the bank.
- Weak audit trails (who changed what, and when).
- Late, manual decisions because analysis and forecasting are not built in.

1.3 Objectives

1. Build one platform that links Budget, Procurement, Payment, Accounting, Inventory and Analytics.

2. Enforce role-based access (RBAC) and data scope in the backend on every request, not only in the UI.

3. Support Maker–Checker (segregation of duties): the person who creates a financial transaction does not approve it.

4. Make financial integrity a guarantee: atomic transactions, no duplicate payments, a full audit log, and no silent edits to posted entries.

5. Provide explainable AI: Cash-Flow & Budget Forecasting, Anomaly Detection & Financial Risk, and an AI Business Assistant that respects the user's permissions.

6. Deliver a working end-to-end demo: create a budget, submit a purchase request, approve it, record receipt, record payment, then see the effect on budget, accounting and inventory.

1.4 Use Case Diagram & Descriptions

Target users to cover as actors: Company Owner/CEO, Finance Manager/CFO, Accountant, Department Manager, Procurement Officer, Inventory/Warehouse Officer, Employee/Requester, Auditor (read-only), External Supplier and Platform Administrator. Payroll/HR, Production and Sales officers belong to later phases.

Diagram of the actors and the Phase 1 (Core MVP) use cases. Payroll/HR, Production and Sales actors belong to later phases, and the External Supplier portal is a Could item (FR-PRO-007), so they are not shown.

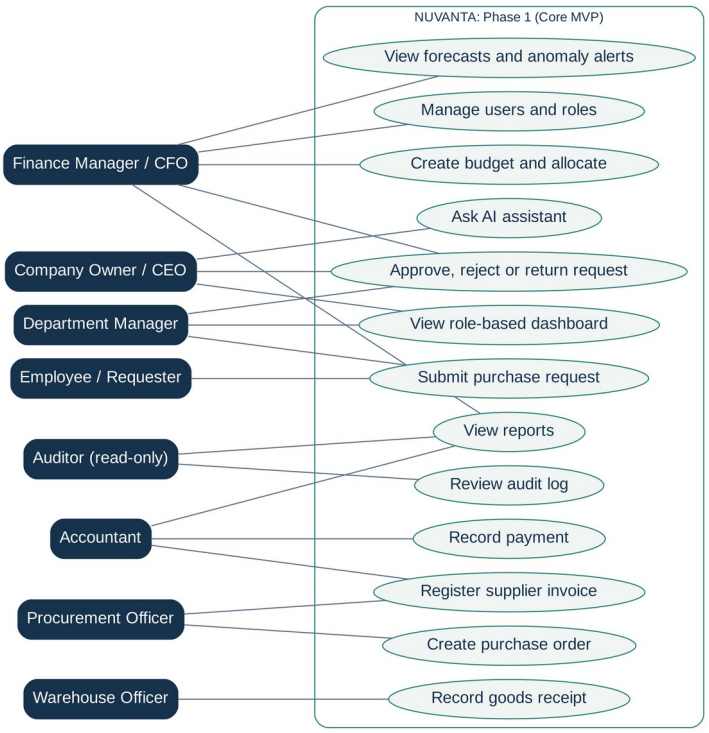


Figure 1.1: Use case diagram, Phase 1

Use case descriptions

Use case	Actor	Goal	Main flow
Manage users and roles	Finance Manager / CFO	Give people the right access	Create user, assign role and department, set data scope; changes are logged.
Create budget and allocate	Finance Manager / CFO	Set the period budget and split it	Create budget with total and opening cash, allocate to departments, cost centers and categories, approve.
Submit purchase request	Employee, Department Manager	Ask to buy something	Enter items, quantity, estimated price and justification, attach files; system checks available budget; submit.
Approve, reject or return request	Department Manager, CFO, Owner	Decide on a request	Open approval inbox, review, decide with a comment; on approval the budget is reserved (Committed).
Create purchase order	Procurement Officer	Order from a supplier	Pick an approved request and a supplier, create and issue the purchase order.
Record goods receipt	Warehouse Officer	Record what was delivered	Select the order, enter received quantities (partial allowed); stock increases and order status updates.
Register supplier invoice	Procurement Officer, Accountant	Capture the supplier bill	Enter invoice linked to order and receipt; mismatches are flagged; invoice is approved.

Use case	Actor	Goal	Main flow
Record payment	Accountant	Pay an approved invoice	Enter payment with idempotency key; on success cash, budget (Committed to Spent) and ledger update in one transaction; on failure the invoice stays unpaid.
View role-based dashboard	Owner, Department Manager and others	See status at a glance	Open the dashboard for the role: budget figures, pending approvals, alerts.
View reports	CFO, Accountant, Auditor	Analyse plan against actual	Filter by date, department, supplier or status; export by permission.
Review audit log	Auditor	Check who did what	Search the read-only audit log by user, date and entity; old and new values are shown.
Ask AI assistant	Owner, CFO	Get answers from data	Ask in natural language; assistant uses permission-checked read tools and shows data source and date.
View forecasts and anomaly alerts	CFO	Act early on risk	Open cash-flow and budget forecasts and flagged invoices with risk score and reason; the AI only suggests.

1.5 Functional & Non-Functional Requirements

Each requirement has a fixed ID (FR-module-number) that is carried into the SRD, the test cases and the traceability matrix. Priority: M Must, S Should, C Could. All Must items belong to Phase 1 (Core MVP). Total: 60 functional requirements; Phase 2–4 features stay as future work.

1.5.1 Functional Requirements

Users, Roles and Access (AUT)

ID	Requirement	Pri.
FR-AUT-001	The system shall authenticate users with secure credentials and protect sessions (JWT or cookies).	M
FR-AUT-002	The system shall let an authorised user create users and assign roles and departments.	M
FR-AUT-003	The system shall check role permissions in the backend on every request, not only hide buttons in the interface.	M
FR-AUT-004	The system shall apply data scope (company, branch, department or own data) to every query.	M
FR-AUT-005	The system shall provide the default roles of the Blueprint and allow permissions to be customised.	S
FR-AUT-006	The system shall enforce Maker–Checker: the creator of a financial transaction shall not approve it, except for documented exceptions.	M
FR-AUT-007	The system shall let users change their password and manage their sessions.	S

Budget and Financial Planning (BUD)

ID	Requirement	Pri.
FR-BUD-001	The system shall let an authorised user create a budget for a financial period with total budget and opening cash.	M
FR-BUD-002	The system shall allocate the budget to departments, cost centers and expense categories, and shall not let allocations exceed the approved total unless a policy allows it with special approval.	M
FR-BUD-003	The system shall reject negative or invalid allocations and log every change with user, date, old and new values.	M
FR-BUD-004	The system shall prevent concurrent updates from exceeding the budget (race condition).	M
FR-BUD-005	The system shall show Planned, Committed, Spent and Available amounts for every allocation.	M
FR-BUD-006	The system shall allow limits per department or category and alert when a set percentage is reached.	S
FR-BUD-007	The system shall allow budget transfers between departments with the proper approval.	S
FR-BUD-008	The system shall compare planned against actual amounts.	S

Purchase Requests and Approvals (REQ)

ID	Requirement	Pri.
FR-REQ-001	The system shall let a user create a purchase request with item, quantity, estimated price, department, cost center, category, justification and attachments.	M
FR-REQ-002	The system shall check available budget when a request is submitted and show the result.	M
FR-REQ-003	The system shall route each request through an approval workflow based on amount and permissions.	M
FR-REQ-004	The system shall let an approver approve, reject or return a request with a comment, and record the decision.	M
FR-REQ-005	The system shall provide an approval inbox listing every item waiting for the user’s decision.	M
FR-REQ-006	The system shall commit budget when a request is approved and release it when the request is rejected, cancelled or closed.	M
FR-REQ-007	The system shall show an activity timeline from creation to closure.	S
FR-REQ-008	The system shall allow approval delegation during absence.	C

Suppliers and Procurement (PRO)

ID	Requirement	Pri.
FR-PRO-001	The system shall manage suppliers and a catalog of products and materials.	M
FR-PRO-002	The system shall create a purchase order from an approved request for a chosen supplier.	M
FR-PRO-003	The system shall record goods receipt against a purchase order and check quantities, including partial deliveries.	M
FR-PRO-004	The system shall register supplier invoices linked to the order and receipt and flag mismatches.	M
FR-PRO-005	The system shall record quotations and compare suppliers.	C

ID	Requirement	Pri.
FR-PRO-006	The system shall handle purchase order cancellation and returns.	C
FR-PRO-007	The system shall give suppliers limited access to their orders, invoices and payment status, without internal budgets.	C

Payments and Cash (PAY)

ID	Requirement	Pri.
FR-PAY-001	The system shall track each payment with the status Pending, Processing, Paid, Failed or Refunded.	M
FR-PAY-002	The system shall support a sandbox payment simulation.	M
FR-PAY-003	The system shall record external transfers with reference number, date and receipt.	M
FR-PAY-004	The system shall process a repeated payment confirmation only once, using an idempotency key and a unique reference constraint.	M
FR-PAY-005	The system shall keep the invoice unpaid when a payment fails, record the reason and allow a retry.	M
FR-PAY-006	The system shall, in one atomic transaction, update cash balance, budget (Committed to Spent) and ledger when a payment succeeds.	M
FR-PAY-007	The system shall manage cash accounts and balances.	M
FR-PAY-008	The system shall not move real money in the first version; real integration only after the provider officially confirms support and legal requirements are checked.	M

Accounting (ACC)

ID	Requirement	Pri.
FR-ACC-001	The system shall provide a basic chart of accounts.	M
FR-ACC-002	The system shall create balanced double-entry ledger entries automatically from financial transactions.	M
FR-ACC-003	The system shall not allow posted entries to be silently edited or deleted; corrections shall use an approved adjusting entry.	M
FR-ACC-004	The system shall provide basic reports (budget vs actual, expenses by department and category, ledger) with PDF or Excel export by permission.	S
FR-ACC-005	The system shall support period closing.	C

Inventory (INV)

ID	Requirement	Pri.
FR-INV-001	The system shall manage items, units of measure and warehouses.	M
FR-INV-002	The system shall increase stock when goods receipt is recorded.	M
FR-INV-003	The system shall record stock issue and transfers between warehouses.	S
FR-INV-004	The system shall alert when stock falls below the minimum level.	S
FR-INV-005	The system shall support stocktaking and stock adjustments.	C

Dashboards, Notifications and Audit (SYS)

ID	Requirement	Pri.
FR-SYS-001	The system shall show role-based dashboards (CEO/CFO, Department Manager, Employee, Warehouse).	M
FR-SYS-002	The system shall record every sensitive action in an audit log (who, when, old and new value) that cannot be silently deleted.	M
FR-SYS-003	The system shall send notifications for pending approvals, failed payments, low stock and budget thresholds.	S
FR-SYS-004	The system shall allow attachments on requests, invoices and receipts.	S
FR-SYS-005	The system shall provide filters by date, department, supplier and status, with exports by permission.	S
FR-SYS-006	The system shall provide global search limited by permissions, and internal comments on requests.	C

AI Features (AI)

ID	Requirement	Pri.
FR-AI-001	The system shall forecast cash flow for the next week, month or quarter and evaluate it against a baseline.	S
FR-AI-002	The system shall estimate the probability that a department will exceed its budget.	S
FR-AI-003	The system shall detect anomalies (duplicate invoices, unusual prices or quantities, invoice–order–receipt differences) using rules first, then a model such as Isolation Forest, and give a risk score with the reason.	S
FR-AI-004	The AI Assistant shall answer natural-language questions using only permission-checked read tools, show the data source and date, and log each query.	S
FR-AI-005	The AI shall only suggest actions; it shall never pay, change a budget or approve a transaction by itself.	M
FR-AI-006	The system shall store AI recommendations and show the data source and date with every AI result.	S

1.5.2 Non-Functional Requirements

Targets come from the KPIs in the Project Planning & Management document. Values marked To be set in SRD need the expected system size (users, data volume) before a number can be fixed.

ID	Category	Requirement	Target / Measurement
NFR-SEC-01	Security	Authentication, session protection and encrypted connections; RBAC and data scope checked in the backend on every request	100% unauthorized access blocked in role-based tests
NFR-SEC-02	Privacy	Sensitive data (salaries, financial details, customer data) is visible only with explicit permission, also through the AI Assistant	100% in permission tests per role
NFR-INT-01	Financial integrity	Atomic transactions, no duplicate payments, accurate budget and ledger figures, balanced entries	100% in reconciliation tests

ID	Category	Requirement	Target / Measurement
NFR-AUD-01	Auditability	Sensitive actions are logged with user, time, old and new values; history cannot be silently deleted	100% of sensitive actions in audit log review
NFR-REL-01	Reliability	Correct handling of payment failure, connection loss and retries; no half-completed postings	Failure and retry test cases pass
NFR-REL-02	Backup and recovery	Regular backups with a tested restore; database scripts and seed data under version control	RTO / RPO: To be set in SRD
NFR-PER-01	Performance	API calls respond quickly under the expected load	Under 2 seconds (API tests and logs); final numbers per data size in SRD
NFR-PER-02	Performance	Dashboards load quickly	Under 3 seconds (browser tools)
NFR-AVL-01	Availability	System is available during working and demo hours	To be set in SRD
NFR-SCA-01	Scalability	Modular monolith design that allows more users, branches and data later	Clear module boundaries (Finance, Procurement, Inventory)
NFR-USA-01	Usability	Clear screens, understandable error messages, role-specific dashboards, easy filters and reports	At least 4 of 5 satisfaction; at least 90% task success
NFR-USA-02	Usability	Responsive interface for desktop and mobile; Arabic and English with suitable date and currency display	Main flows usable on both screen sizes; language support is Should
NFR-MNT-01	Maintainability	Separated modules, documented APIs (OpenAPI), automated tests, code reviews on every pull request	Unit coverage at least 70% on financial and permission logic; 100% PRs reviewed
NFR-TST-01	Testability / Quality	Requirements are covered by test cases and tests run in CI	At least 90% requirements covered; pass rate at least 95%; 0 critical bugs at delivery
NFR-AIG-01	AI governance	AI results are explainable (source and date), recommendations are logged, AI respects user permissions and always keeps human approval	100% in permission tests; MAPE for forecasting; precision and recall for anomalies

1.6 Software Architecture

The system is a **modular monolith** with clear modules (Finance, Procurement, Inventory and so on). The frontend is an Angular application that talks to an ASP.NET Core Web API; data is stored in SQL Server through Entity Framework Core; a separate Python (FastAPI) service provides the AI features. Security uses ASP.NET Core Identity with JWT/cookies and role/policy authorization.

The frontend talks to the ASP.NET Core Web API, which is a modular monolith. Security (authentication, RBAC and data scope) is checked in the backend on every request. Data is stored in SQL Server through Entity Framework Core. The Python AI service is separate: the API sends it forecast and anomaly requests, and the AI Assistant reads data only through permission-checked read tools using the user's token.

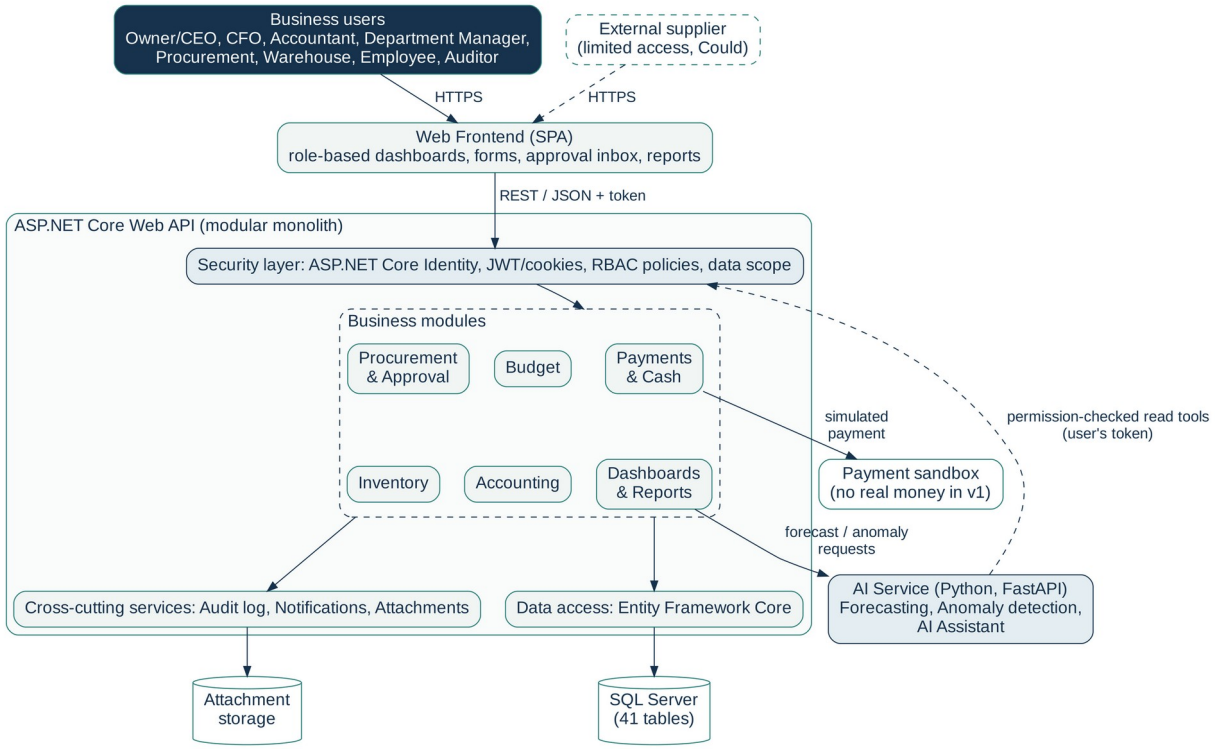


Figure 1.2: High-level architecture

The frontend is the Angular application described in section 4.

2. Database Design & Data Modeling

The database has **41 tables** in **8 functional groups**: Company & User Management (8), Financial Planning & Budgeting (7), Procurement & Purchasing (7), Inventory Management (5), Supplier Invoices & Payments (4), Accounting & Financial Reporting (5), Approval Workflow (4) and Audit & Tracking (1).

2.1 ER Diagram

The ERD uses Chen notation: rectangles are entities, diamonds are relationships, ovals are attributes, and each relationship line shows its cardinality (1, N, 0..1). The model is split into 8 pages, one per group. The editable source is the draw.io file; the images below are exports of its pages.

ERD link: Open the ERD in draw.io

2.1.1 Group 1: Company & User Management

Company is the root entity. It operates branches, houses departments (which can have sub-departments) and employs users. Access control follows RBAC: users get roles through USER_ROLE, and roles get permissions through ROLE_PERMISSION.

2.1.2 Group 2: Financial Planning & Budgeting

A company defines financial periods, cost centers and expense categories. A budget belongs to a period and is split into allocations by department, cost center and category. Transfers move money between two allocations, and reservations hold allocated money for a purchase request.

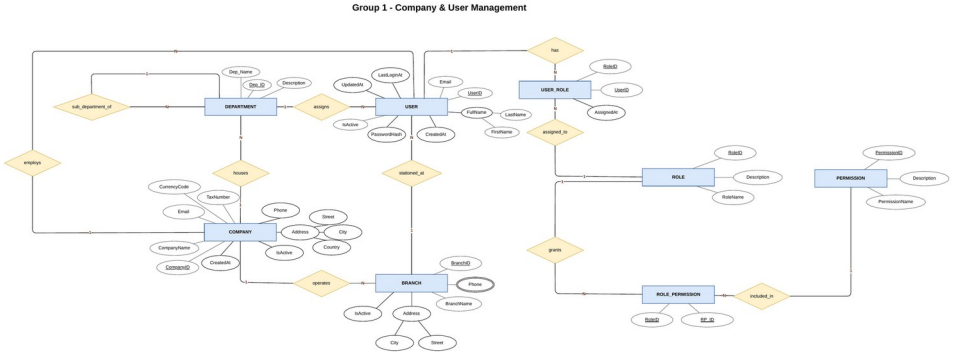


Figure 2.1: Group 1: Company & User Management

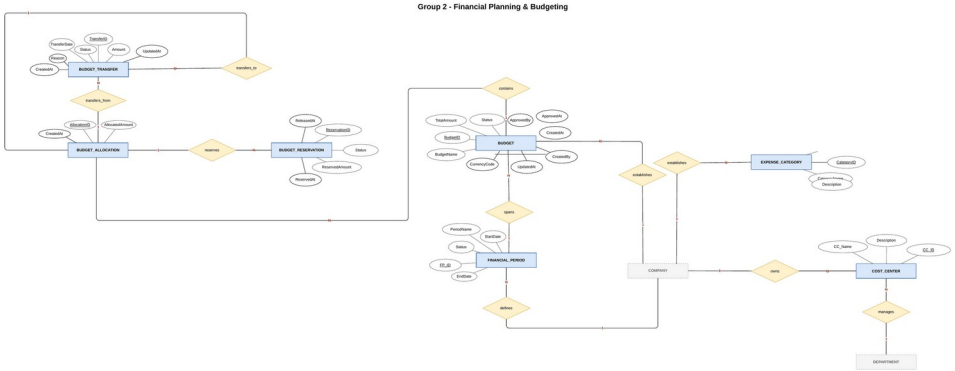


Figure 2.2: Group 2: Financial Planning & Budgeting

2.1.3 Group 3: Procurement & Purchasing

The procure-to-receive flow: a purchase request (with items) is converted into a purchase order for a supplier, and goods receipts record what is actually delivered against each purchase order item.

2.1.4 Group 4: Inventory Management

Inventory items belong to categories and are stored in warehouses of a branch. StockMovements is the source of truth; InventoryBalances keeps the current quantity per item and warehouse.

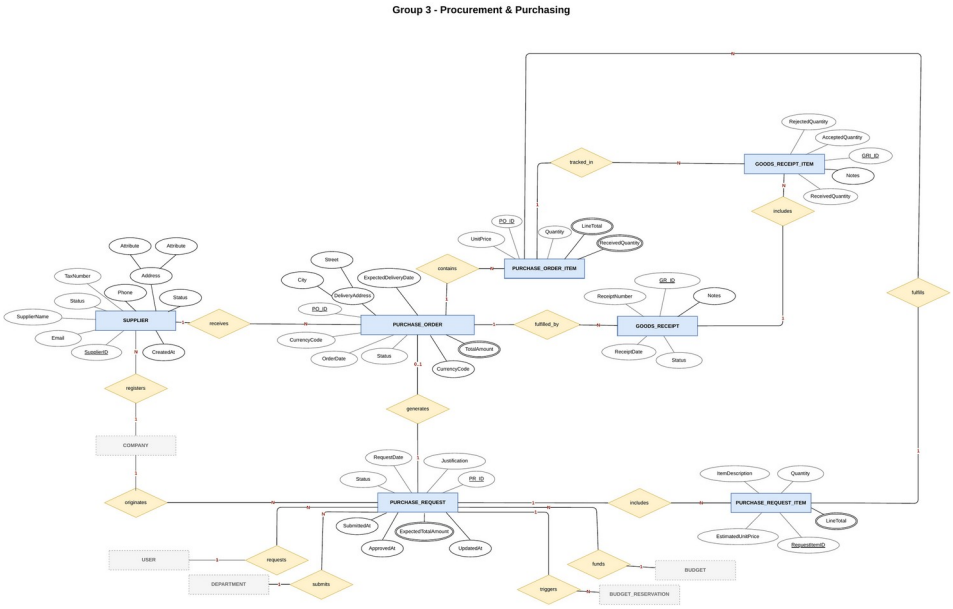


Figure 2.3: Group 3: Procurement & Purchasing

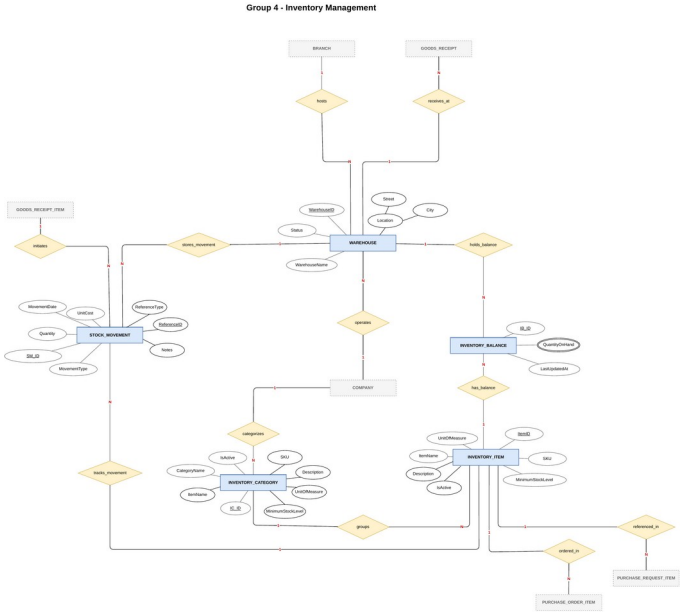


Figure 2.4: Group 4: Inventory Management

2.1.5 Group 5: Supplier Invoices & Payments

A supplier invoice (with items) is linked to a purchase order. Payments are made to a supplier and allocated to one or more invoices through PaymentAllocations.

2.1.6 Group 6: Accounting & Financial Reporting

A hierarchical chart of accounts, ledger entries with debit and credit lines inside a financial period, and cash or bank accounts whose transactions are generated by payments.

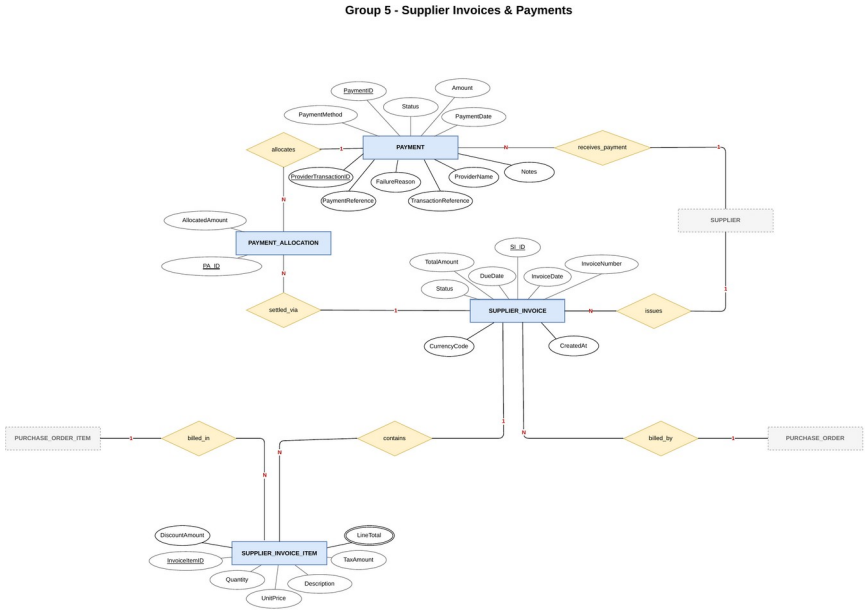


Figure 2.5: Group 5: Supplier Invoices & Payments

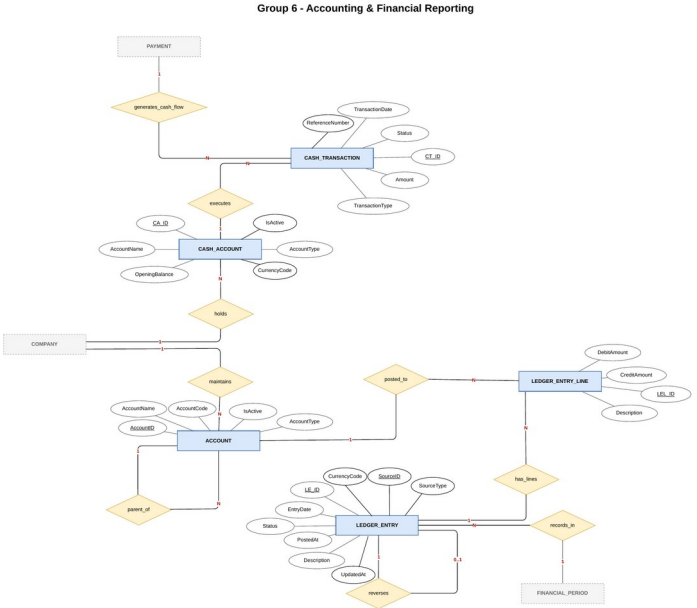


Figure 2.6: Group 6: Accounting & Financial Reporting

2.1.7 Group 7: Approval Workflow

Rules decide who must approve an entity by amount range. A request is split into ordered steps, and every decision is logged as an action.

2.1.8 Group 8: Audit & Tracking

Every important action by a user inside a company is recorded in AuditLogs with old and new values.

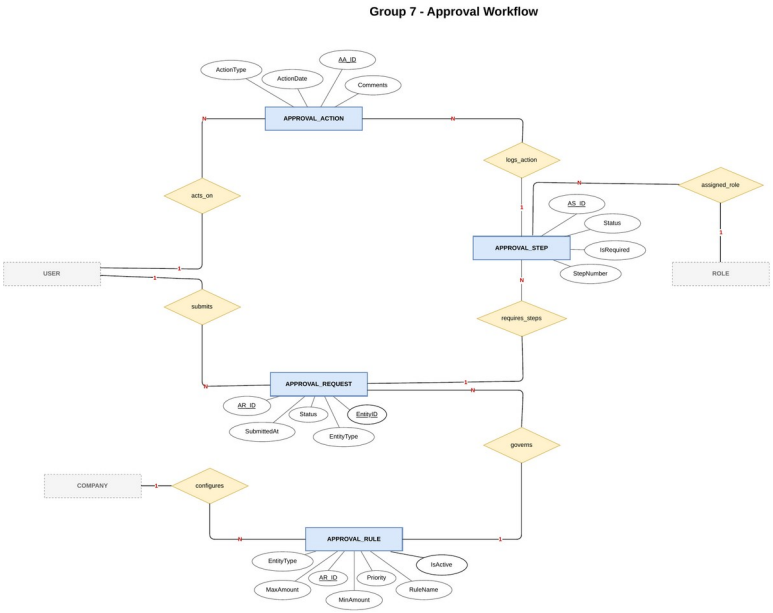


Figure 2.7: Group 7: Approval Workflow

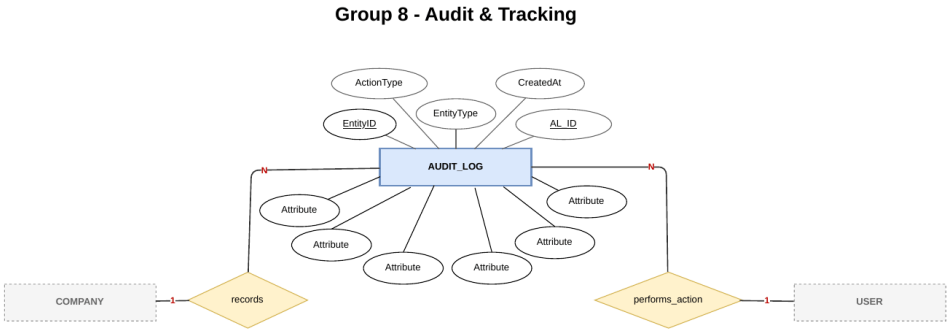


Figure 2.8: Group 8: Audit & Tracking

2.2 Logical & Physical Schema

The schema translates the ERD into tables with primary keys (PK) and foreign keys (FK). Each table box lists its columns with a data type, and relationship lines connect FK and PK columns.

Schema link: Open the Schema in draw.io

The full diagram is large (41 tables with all relationship lines), so it is not embedded as an image. Open it through the link above and zoom into any table; section 2.3 lists the key relationships in text form. The full column list, types and rules of every table are in NUV-S1-03 (Database Constraints & Data Rules).

2.3 Keys and Relationships

The main foreign keys of the schema, grouped by module. Optional links are marked (optional).

Parent	Child	Foreign key / note
Companies	Branches, Departments, Users, Roles	CompanyID (Roles: system-wide roles have no company)
Branches	Departments (optional), Users (optional), Warehouses	BranchID
Departments	Departments (self)	ParentDepartmentID (optional); a department cannot be its own parent
Users, Roles	UserRoles (many-to-many)	UserID + RoleID (composite PK)
Roles, Permissions	RolePermissions (many-to-many)	RoleID + PermissionID (composite PK)
FinancialPeriods	Budgets, LedgerEntries	FinancialPeriodID
Budgets	BudgetAllocations, PurchaseRequests	BudgetID
Departments, CostCenters, ExpenseCategories	BudgetAllocations	DepartmentID, CostCenterID, CategoryID (at least one is set)
BudgetAllocations	BudgetTransfers	FromAllocationID, ToAllocationID (must differ)
BudgetAllocations, PurchaseRequests	BudgetReservations	AllocationID, PurchaseRequestID
PurchaseRequests	PurchaseRequestItems, PurchaseOrders	PurchaseRequestID
Suppliers	PurchaseOrders, SupplierInvoices, Payments	SupplierID
PurchaseOrders	PurchaseOrderItems, GoodsReceipts, SupplierInvoices	PurchaseOrderID
PurchaseRequestItems	PurchaseOrderItems	RequestItemID
GoodsReceipts, Warehouses	GoodsReceiptItems, GoodsReceipts	GoodsReceiptID, WarehouseID
PurchaseOrderItems	GoodsReceiptItems, SupplierInvoiceItems	PurchaseOrderItemID
InventoryCategories	InventoryItems	InventoryCategoryID
InventoryItems, Warehouses	StockMovements, InventoryBalances	ItemID, WarehouseID
GoodsReceiptItems	StockMovements	GoodsReceiptItemID
SupplierInvoices	SupplierInvoiceItems, PaymentAllocations	SupplierInvoiceID

Parent	Child	Foreign key / note
Payments	PaymentAllocations, CashTransactions	PaymentID
CashAccounts	CashTransactions	CashAccountID
Accounts	Accounts (self), LedgerEntryLines	ParentAccountID (optional), AccountID
LedgerEntries	LedgerEntryLines, LedgerEntries (reversal)	LedgerEntryID, ReversedByEntryID (optional)
ApprovalRules	ApprovalRequests	ApprovalRuleID
ApprovalRequests	ApprovalSteps	ApprovalRequestID
ApprovalSteps	ApprovalActions	ApprovalStepID
Roles, Users	ApprovalSteps	RequiredRoleID, AssignedTo
Users, Companies	AuditLogs	UserID, CompanyID

2.4 Physical Design Rules

The full per-table specification (columns, types, unique constraints, CHECK rules) is in **NUV-S1-03 Database Constraints & Data Rules**. The main points are summarised here.

- **Inventory source of truth:** StockMovements. InventoryBalances is updated in the same transaction as every movement and can be rebuilt from movements.
- **BudgetReservations** is an independent table. Available amount = AllocatedAmount minus Active reservations minus Consumed reservations.
- **PurchaseRequests.BudgetID** is kept; the application checks that a reserved allocation belongs to the same budget.
- **Payments:** PaymentReference (internal number) and ProviderTransactionID (external id) are kept; TransactionReference is removed.
- **Ledger balance rule:** total debit equals total credit for a Posted entry, enforced at posting time with a trigger as a safety net.
- **No hard delete** for financial or operational records; Status or IsActive is used instead.

2.5 Normalization Considerations

The design was checked against the first three normal forms. The result per rule and the deliberate exceptions are listed below.

Normal form	Result	How the design meets it
1NF	Met	Every column holds a single value. Addresses are flattened into Street, City and Country; repeating data such as order lines, invoice lines and role assignments is stored in child tables, not in lists. The only structured values are the JSON old/new values of AuditLogs, which are kept as audit snapshots and never queried as columns.
2NF	Met	Tables with a single-column key cannot have a partial dependency. The tables with a composite key are the junction tables UserRoles (UserID, RoleID) and RolePermissions (RoleID, PermissionID); their only extra column, AssignedAt, depends on the whole key.

Normal form	Result	How the design meets it
3NF	Met, with the exceptions below	Descriptive data lives in its own table and is referenced by key: roles, permissions, categories, suppliers, items, warehouses, accounts. No column depends on another non-key column, for example the supplier name is not copied into orders or invoices.

3. Data Flow & System Behavior

3.1 DFD

Context diagram (level 0)

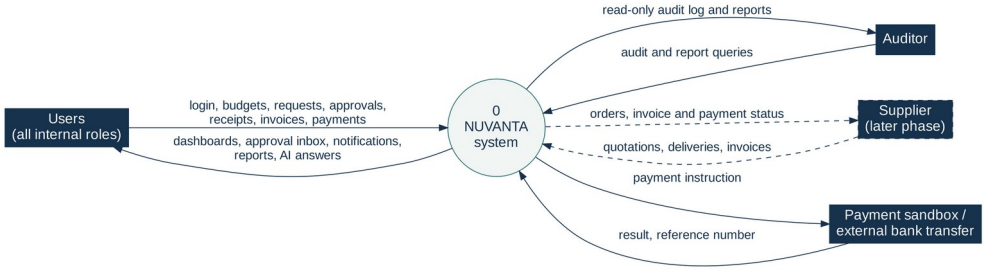


Figure 3.1: Context diagram

Level 1

The eight main processes, the nine data stores and the main data flows. Dashed lines show writing to the audit log.

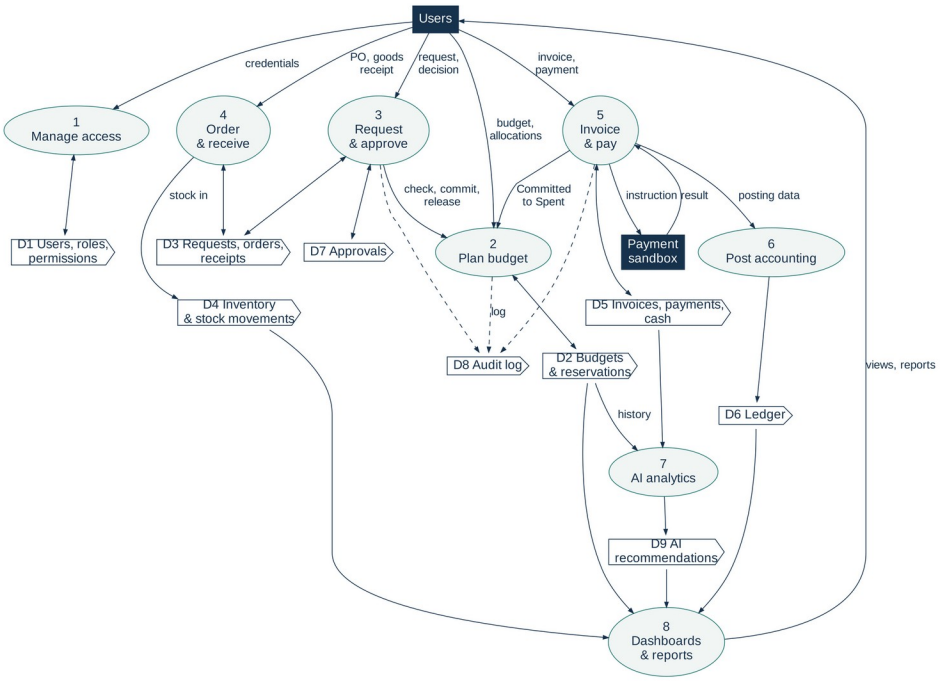


Figure 3.2: Data flow diagram, level 1

Level 2

Level 2 breaks down the two processes that carry the most business rules: process 3 (Request & approve) and process 5 (Invoice & pay). Process numbers follow level 1 (3.1 to 3.5 and 5.1 to 5.5). Dashed lines show writing to the audit log.

Process 3: Request & approve

Process 5: Invoice & pay

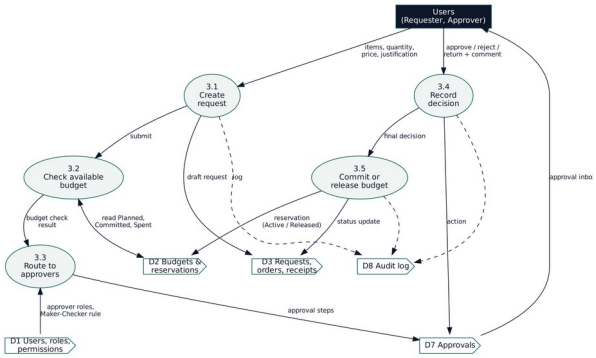


Figure 3.2a: Data flow diagram, level 2, process 3

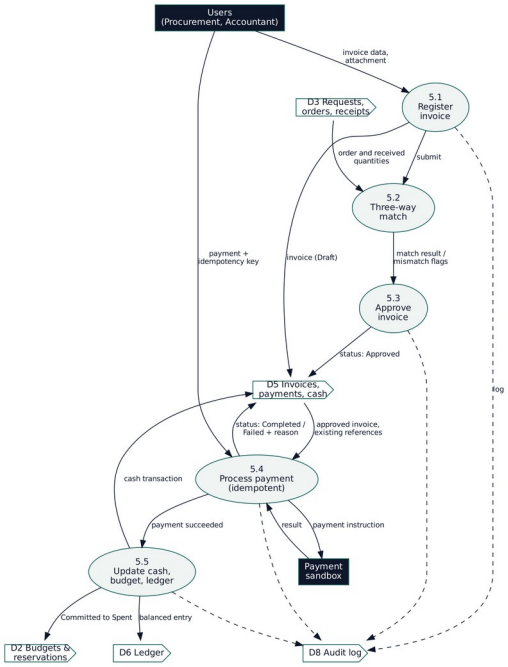


Figure 3.2b: Data flow diagram, level 2, process 5

3.2 Sequence Diagrams

Four key interactions. Each financial change happens in one atomic transaction and is written to the audit log.

Submit a purchase request

Approve a purchase request

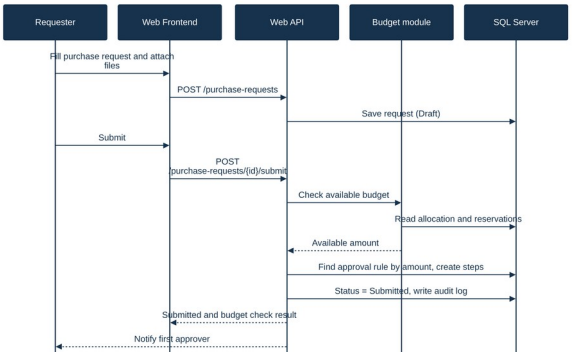


Figure 3.3: Sequence, submit a purchase request

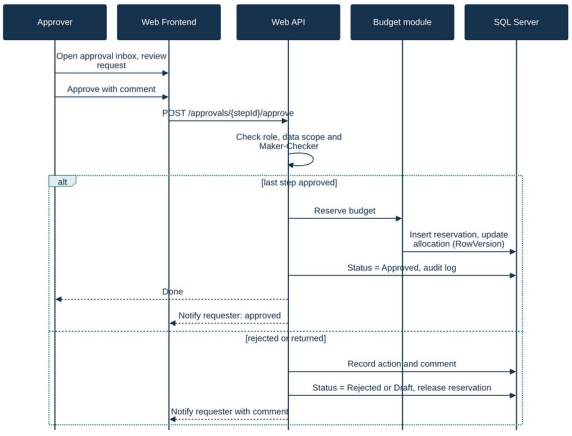


Figure 3.4: Sequence, approve or reject a purchase request

Goods receipt

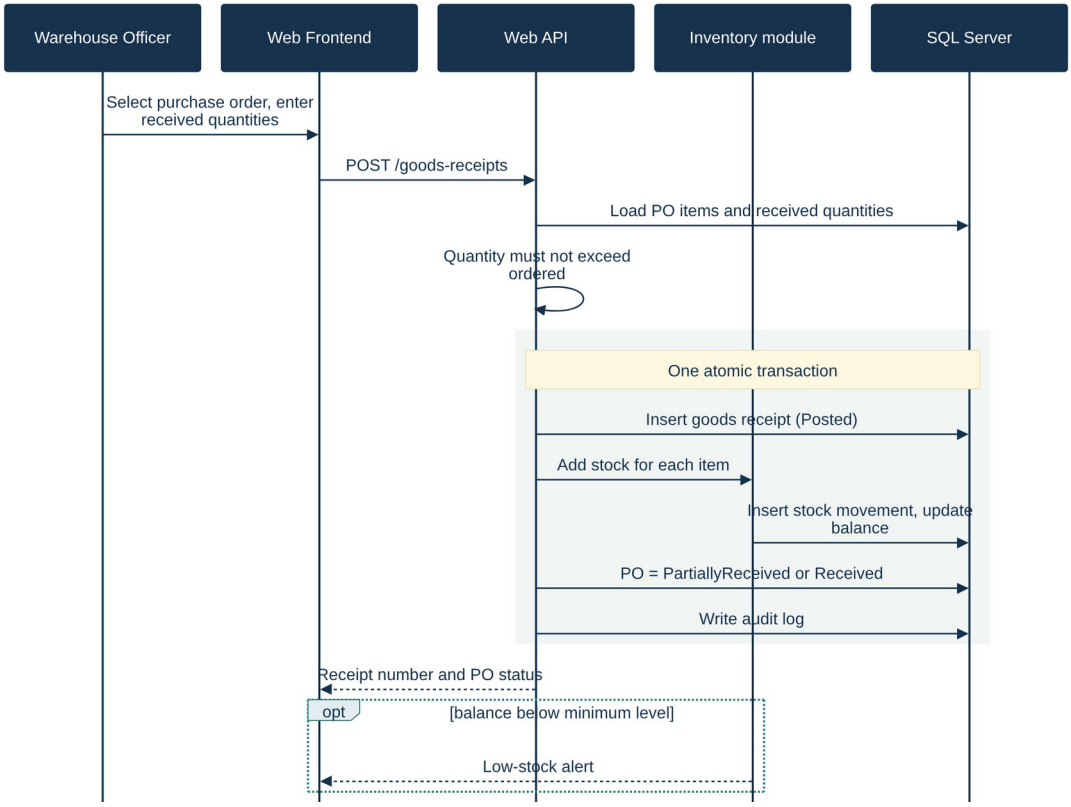


Figure 3.5: Sequence, goods receipt

Payment

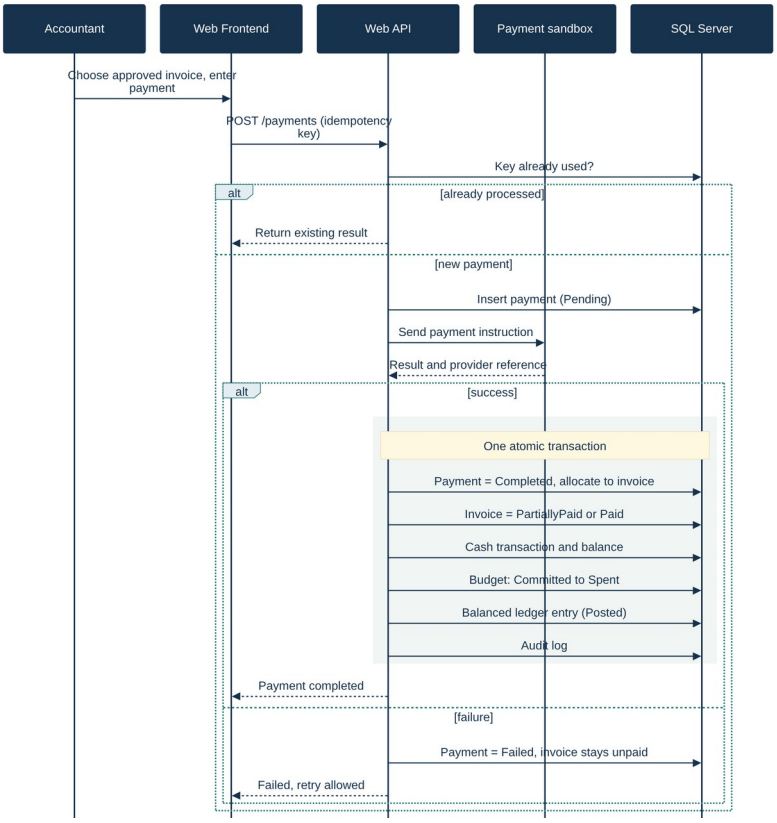


Figure 3.6: Sequence, payment recording

3.3 Activity Diagram

The main process from budget to payment, in two parts.

Part A: budget to purchase order

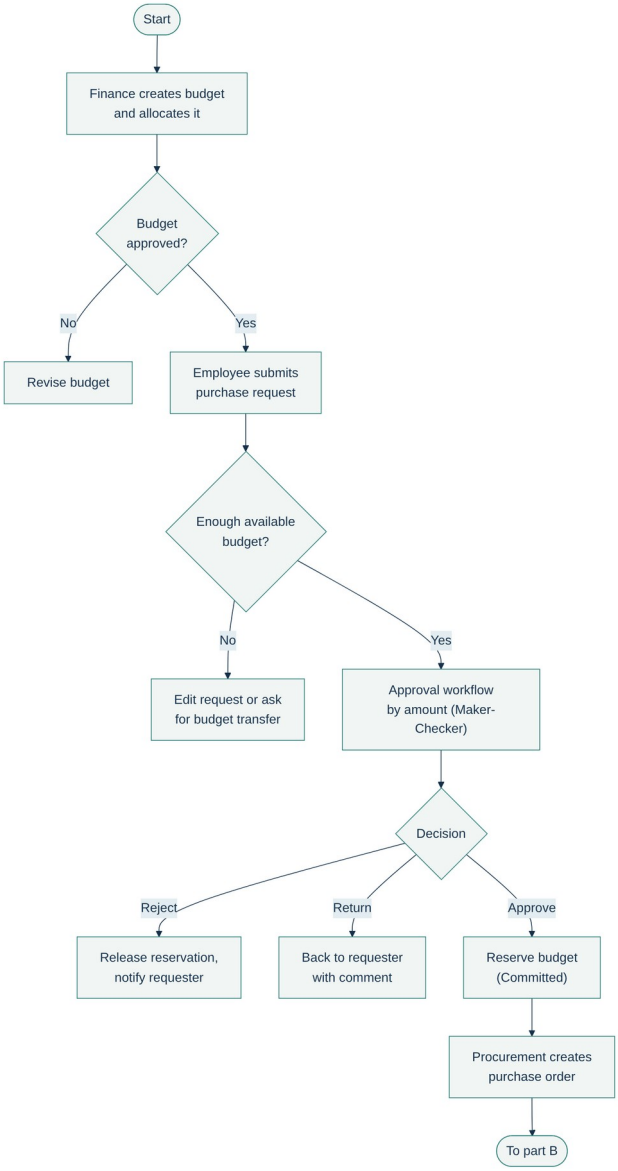


Figure 3.7: Activity diagram, part A

Part B: goods receipt to payment

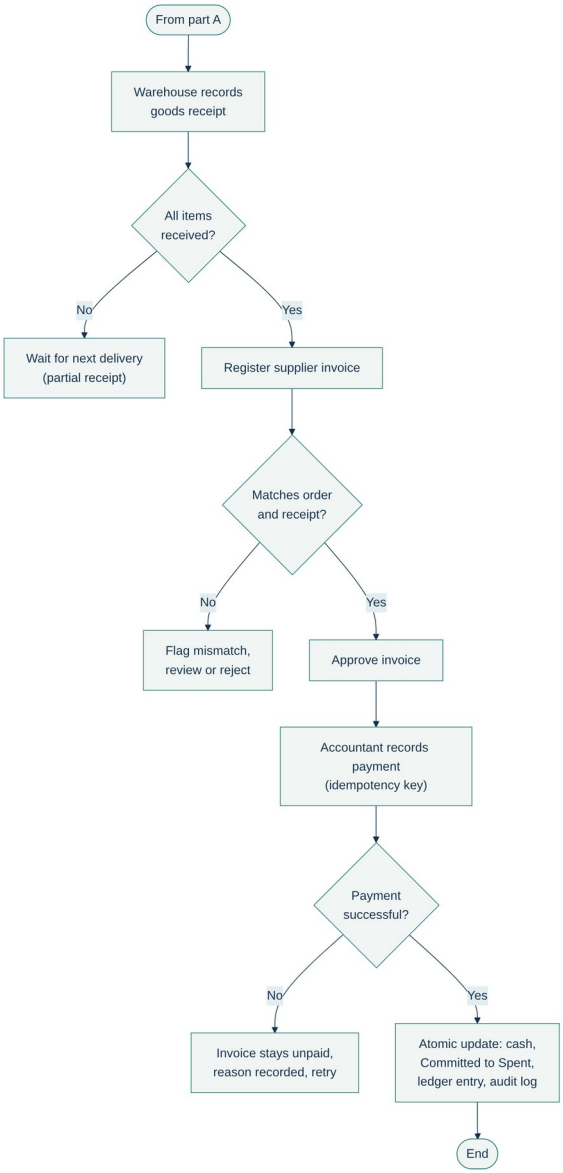


Figure 3.8: Activity diagram, part B

3.4 State Diagram

States use the allowed values of section 2.4. The transitions are proposed from the process rules and must be checked against NUV-S1-03, section 4.2.

PurchaseRequest

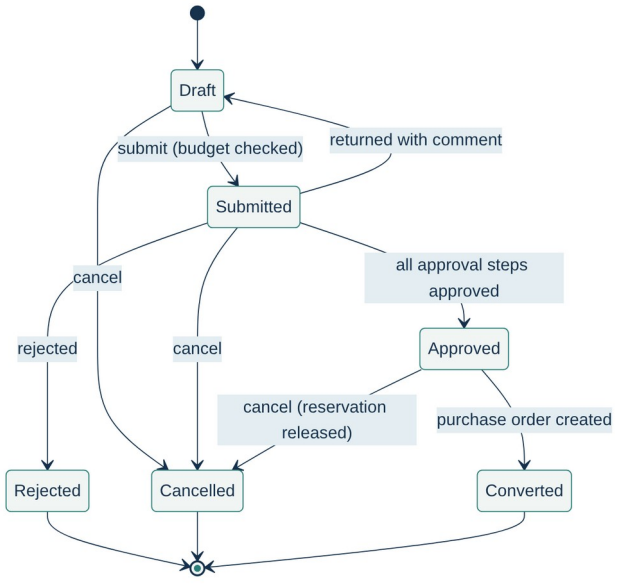


Figure 3.9: State diagram, PurchaseRequest

SupplierInvoice

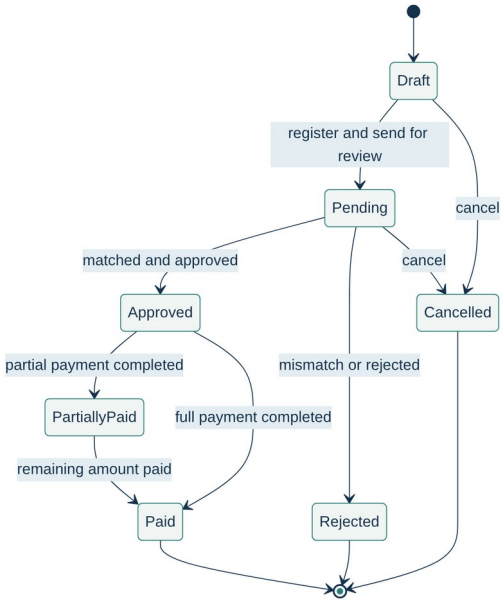


Figure 3.10: State diagram, SupplierInvoice

Payment

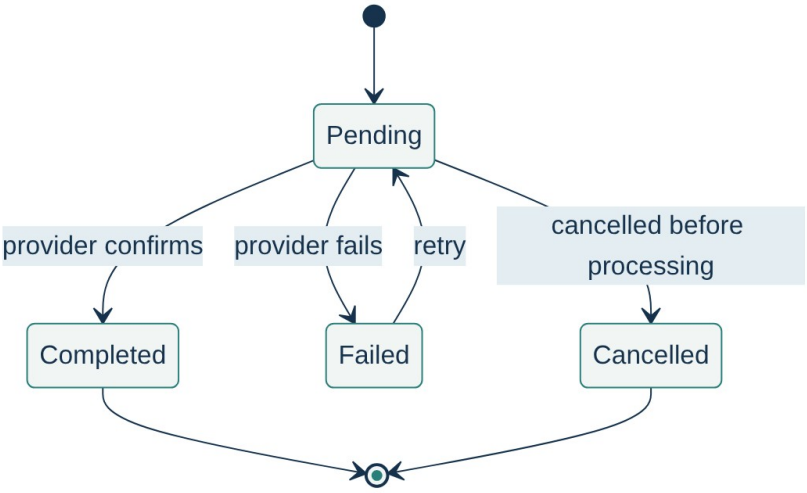


Figure 3.11: State diagram, Payment

LedgerEntry

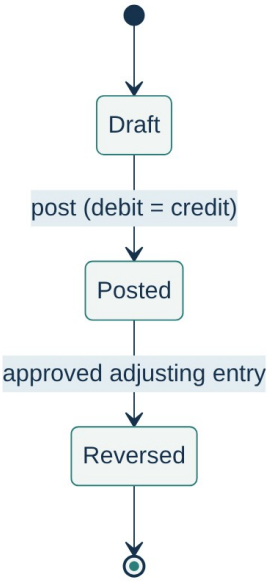


Figure 3.12: State diagram, LedgerEntry

Payment states follow NUV-S1-03 (Pending, Completed, Failed, Cancelled). FR-PAY-001 lists Pending, Processing, Paid, Failed, Refunded.

3.5 Class Diagram

Main domain classes in five parts. They are simplified: only the key attributes and methods are shown, and the full column list is in the physical schema. Class names follow the table names.

Company, access, approval and audit

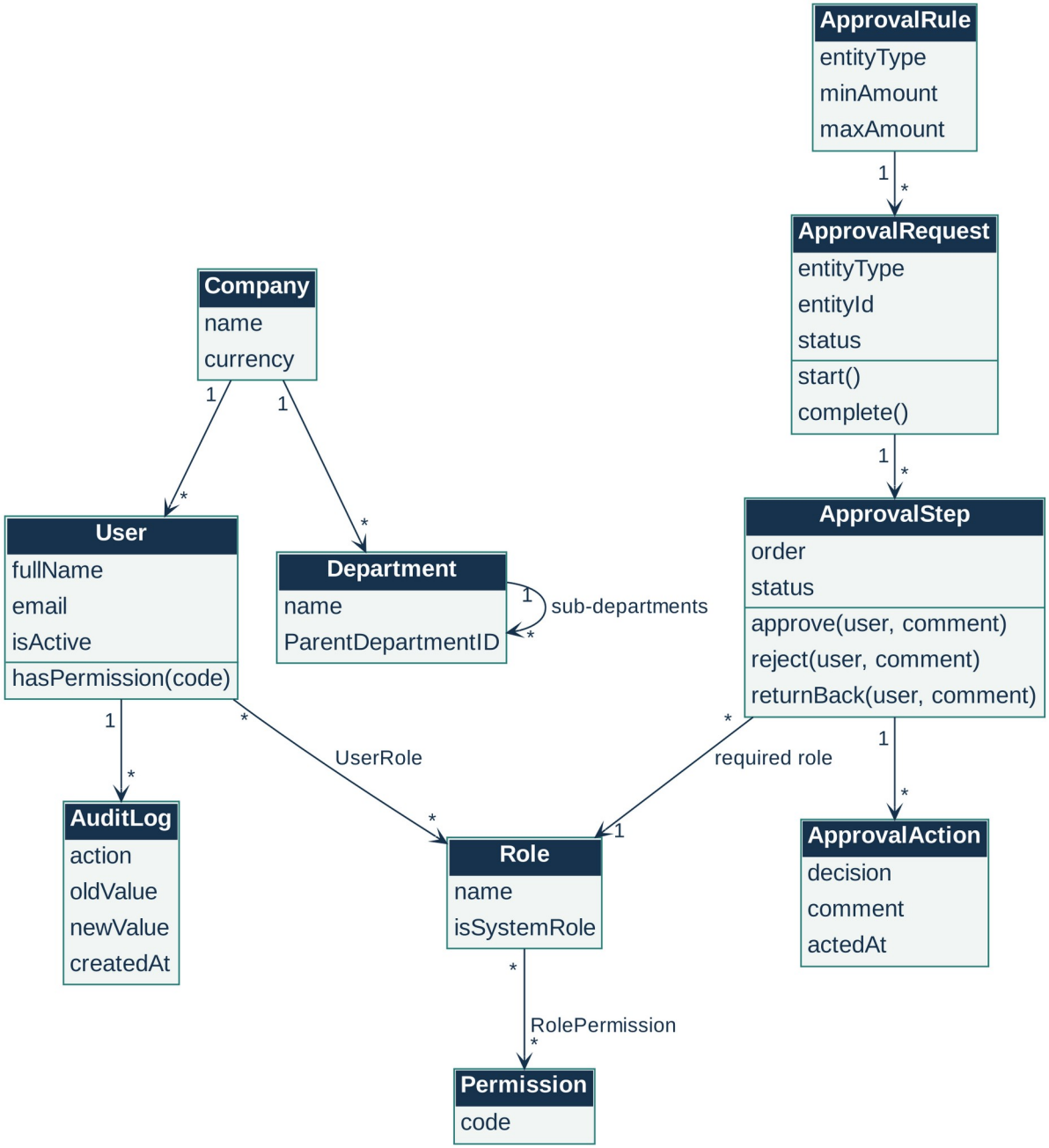


Figure 3.13: Class diagram, access, approval and audit

Budget

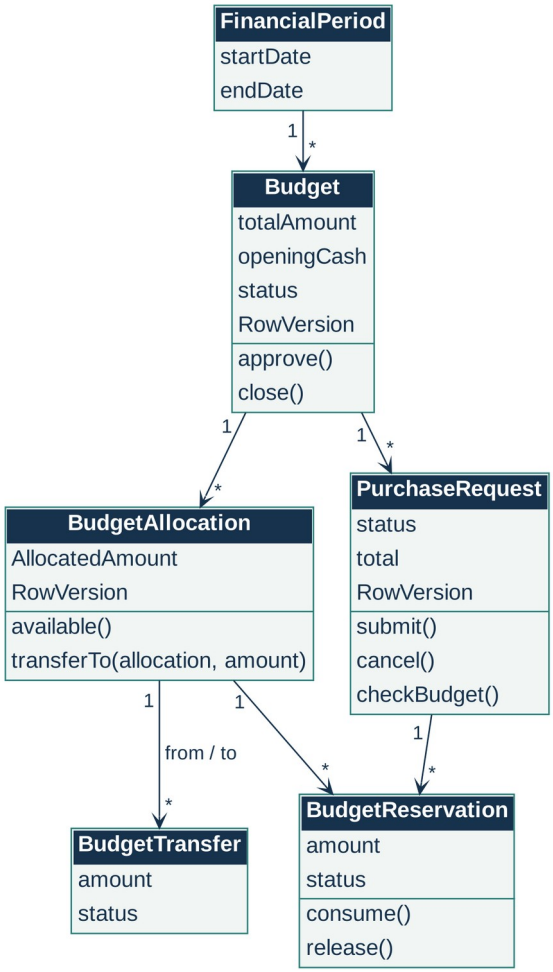


Figure 3.14: Class diagram, budget

Procurement

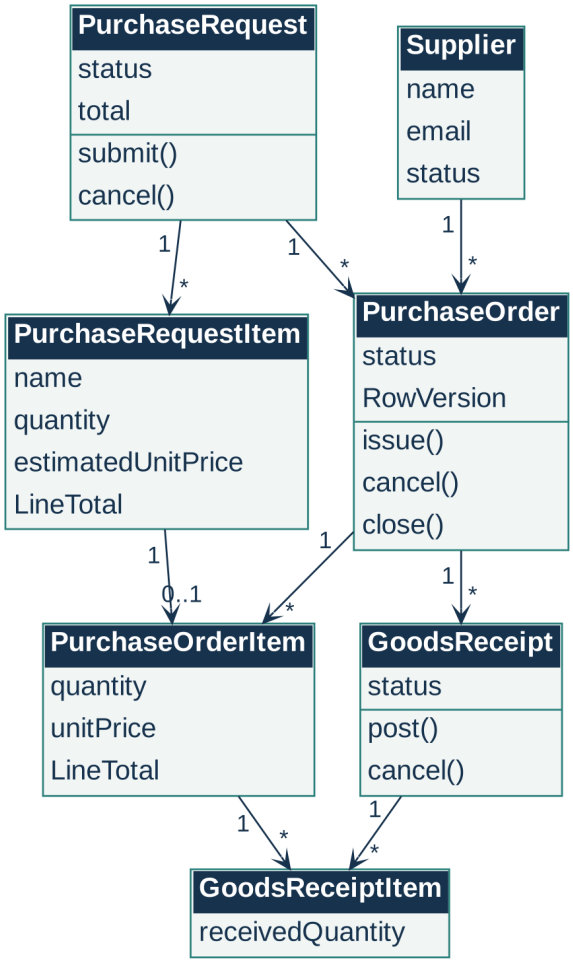


Figure 3.15: Class diagram, procurement

Invoices and payments

Accounting and inventory

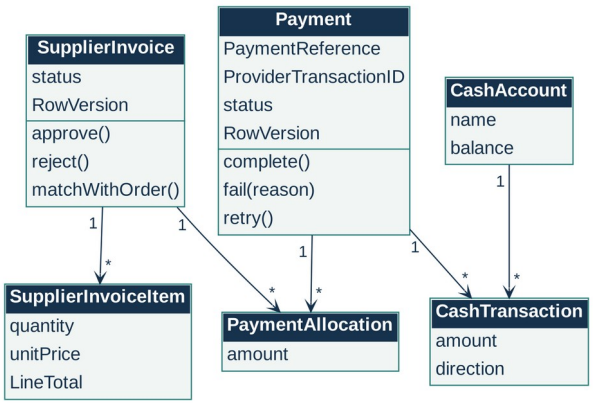


Figure 3.16: Class diagram, invoices and payments

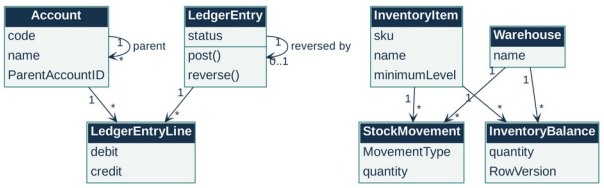


Figure 3.17: Class diagram, accounting and inventory

4. UI/UX Design & Prototyping

4.1 Wireframes & Mockups

The screens below come from the NUVANTA frontend, an Angular web application. Each screen sits inside a shared application layout (MainLayout) with navigation to every module. This is the basic version of the frontend. It is still under development and will be extended further. It currently uses sample data and is not yet connected to the backend.

Login

The entry screen of the application.

Dashboard

A quick view of the company's financial and operational position: budget summary cards (Total Budget, Committed, Spent, Available), a budget versus spending chart across six months, recent transactions, pending approvals and inventory alerts for low-stock and out-of-stock items.

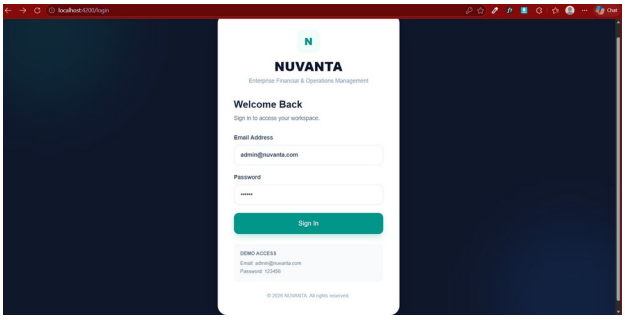


Figure 4.1: Login page

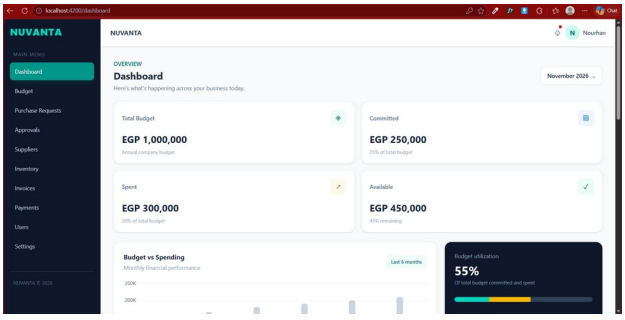


Figure 4.2: Dashboard, budget summary cards

Approvals

Submitted requests for review, with request ID, item, requester, amount, department, submission date and current status.

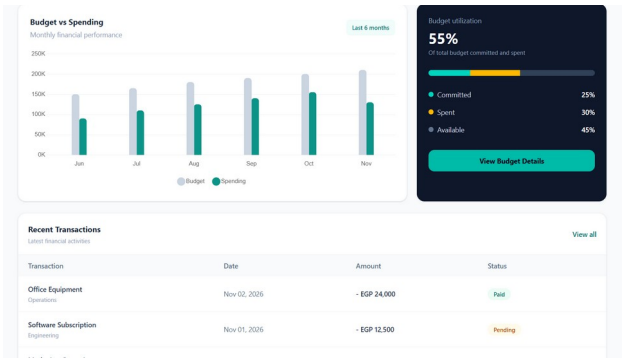


Figure 4.3: Dashboard, budget vs. spending chart and recent transactions

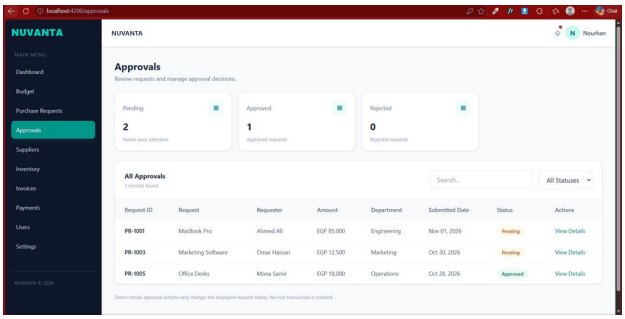


Figure 4.4: Approvals page

Invoices

Supplier invoices with amounts, due dates and payment states (Paid, Pending, Overdue). The figure shows the narrow-screen responsive layout.

User Management

Team members with name, email, role, department and account status. Roles are Admin, Manager, Requester, Accountant and Auditor. The figure shows the Add New Record form.

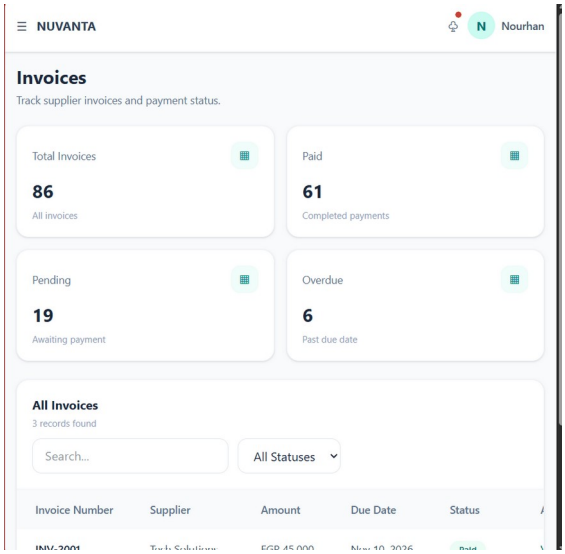


Figure 4.5: Invoices page (responsive, narrow-screen layout)

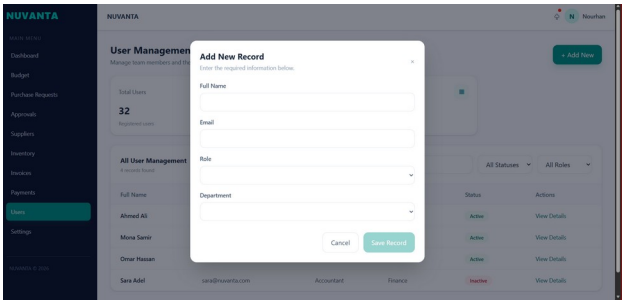


Figure 4.6: User Management, Add New Record form

Settings

Profile, company information, currency, time zone, email notifications, approval reminders and low-stock alerts.

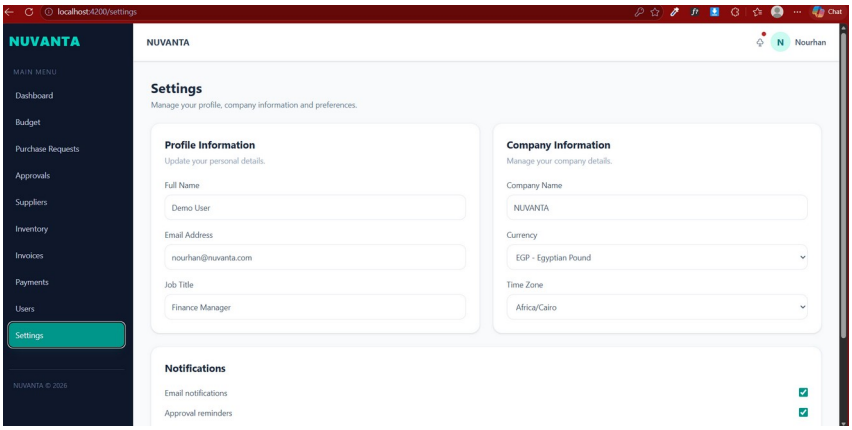


Figure 4.7: Settings page

Wireframes of planned screens

The frontend does not yet have these screens. The low-fidelity wireframes below use the same layout and colors as the current interface (Figures 4.1 to 4.7) and show the screens needed for the demo flow of objective 6, plus the audit log and the AI assistant. They are layout proposals with sample data.

Create budget and allocate

Budget details (name, period, total, opening cash) and the allocation table by department, cost center and category. The screen shows allocated and unallocated amounts and blocks allocations above the total (FR-BUD-001, FR-BUD-002).

New purchase request

Request form with items, justification and attachments. The side panel shows Allocated, Committed, Spent and Available for the chosen budget and the approval route, so the requester sees the budget check before submitting (FR-REQ-001, FR-REQ-002).

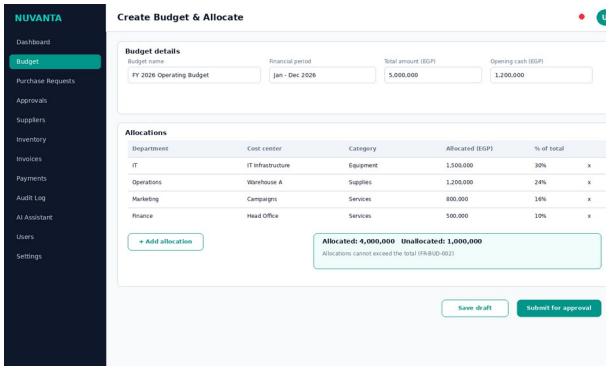


Figure 4.8: Wireframe, create budget and allocate

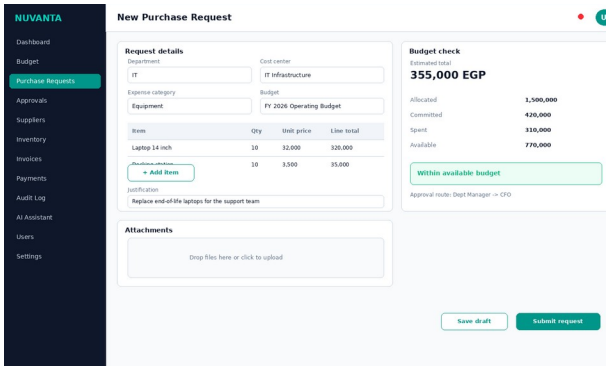


Figure 4.9: Wireframe, new purchase request

Record goods receipt

Receipt against a purchase order with ordered, previously received, received, accepted and rejected quantities. Partial delivery is allowed, and stock increases by the accepted quantity (FR-PRO-003, FR-INV-002).

Audit log

Read-only search by user, entity and date. Each row opens the full old and new values (FR-SYS-002).

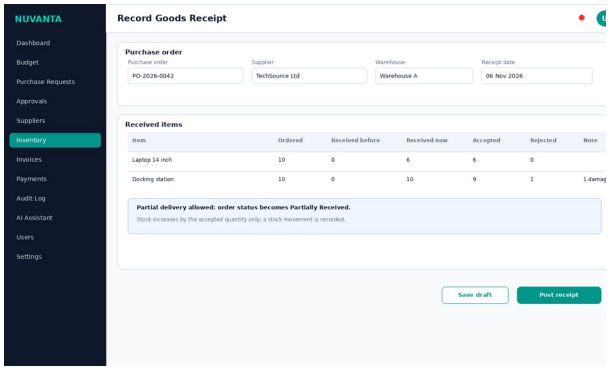


Figure 4.10: Wireframe, record goods receipt

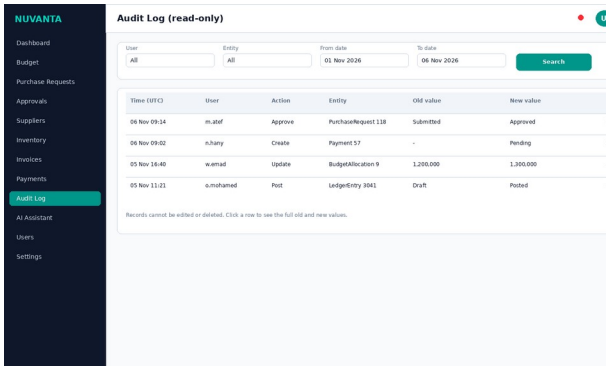


Figure 4.11: Wireframe, audit log

AI assistant

Chat screen. Every answer shows the data source and the date of the data, and states that the AI only suggests (FR-AI-004 to FR-AI-006).

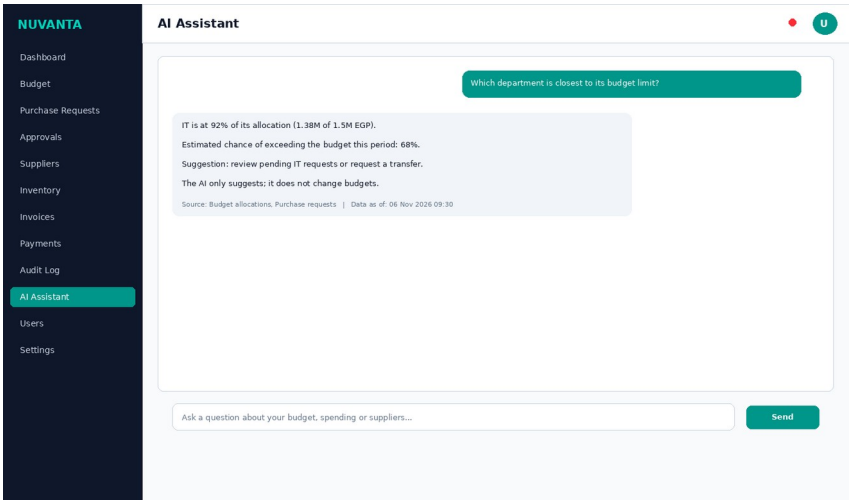


Figure 4.12: Wireframe, AI assistant

4.2 UI/UX Guidelines

The guidelines below describe how the current basic frontend is designed and built. They will be updated as the frontend is developed further.

- **Consistent layout:** All internal pages share one application shell (MainLayout) with the same navigation, so users find the same structure on every screen.
- **Reusable page pattern:** Budget, purchase requests, approvals, suppliers, inventory, invoices, payments, users and settings use one shared page component (ModulePage) driven by route data. This keeps titles, metrics, tables and forms consistent across modules.
- **Dashboard first:** The dashboard is the default internal page (the root path and unknown routes redirect to it) and summarizes budget figures, spending, transactions, pending approvals and inventory alerts. Unknown routes redirect to it.
- **Search and filtering:** Every record list supports text search and status filtering; the users module also supports role filtering.
- **Status indicators:** Record states (for example Paid, Pending, Overdue, In Stock, Low Stock, Out of Stock) are shown as clear status labels so they can be scanned quickly.
- **Responsive design:** Layout, spacing and colors are styled with Tailwind CSS, and the navigation adapts to smaller screens (see Figure 4.5).
- **Data presentation:** Currency values are formatted in EGP. The dashboard chart is drawn with Chart.js.
- **Forms:** Entry forms are shown on demand, and calculated values such as the estimated request amount are derived from the values entered.
- **Accessibility:** Accessibility improvements and responsive testing across browsers and devices are planned as future enhancements.

Note: validation, role permissions and business rules shown in the UI must also be enforced by the backend on every request.

Typography and spacing

- One sans-serif family for the whole application (the default Tailwind sans-serif stack, to be confirmed by the frontend team); Arabic needs a font with Arabic support such as Noto Sans Arabic.

- Type scale: page title 24 to 28 px semibold; section title 18 px; card value 24 px semibold; body 14 px; small labels 12 px. Section labels such as OVERVIEW use small uppercase letters in the teal color.
- Spacing on a 4 px grid; cards use 16 to 24 px padding; tables use compact rows with a clear header.

Accessibility rules (proposed)

- Text contrast of at least 4.5 to 1 (3 to 1 for large text); do not show state by color alone: every status has a text label as well as a color.
- Everything usable with the keyboard, with a visible focus outline; buttons and links have clear names, icons have text alternatives.
- Every form field has a label; error messages say what is wrong and how to fix it, and are linked to the field.
- Tables on phones scroll horizontally or collapse into cards; touch targets are at least 44 px.
- Arabic and English: the layout switches to right-to-left for Arabic, and dates and amounts follow the language setting (Should, NFR-USA-02).
- Test with a keyboard and with Lighthouse or axe before each milestone.

5. System Deployment & Integration

5.1 Technology Stack

Layer	Technology
Frontend	Angular 22 (standalone components), TypeScript, Tailwind CSS, Chart.js
Backend	ASP.NET Core Web API, C#, Entity Framework Core
Database	SQL Server
AI Service	Python, FastAPI, pandas, scikit-learn, LLM integration
Security & Infrastructure	ASP.NET Core Identity, JWT/Cookies, Role/Policy authorization, Docker, CI/CD, logging and monitoring
Architecture	Modular Monolith (clear modules: Finance, Procurement, Inventory, and so on)

5.2 Deployment Diagram

The system runs as containers behind a reverse proxy: the frontend, the API and the AI service. The database runs on its own server with scheduled backups. The payment provider is only a sandbox in the first version.

The hosting platform (cloud or on-premises) is not decided yet and is not shown.

5.3 Component Diagram

The API is split into business modules with shared services. The approval engine is used by procurement, and the payment module updates the budget and posts to accounting in the same transaction. The AI service has three components and only reads data through the API.

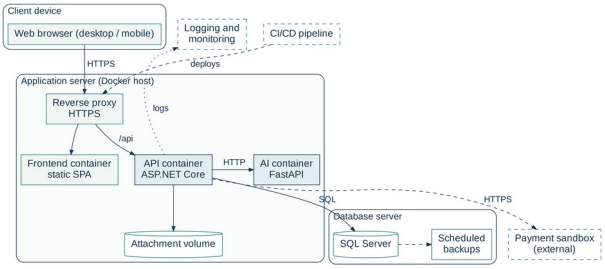


Figure 5.1: Deployment diagram

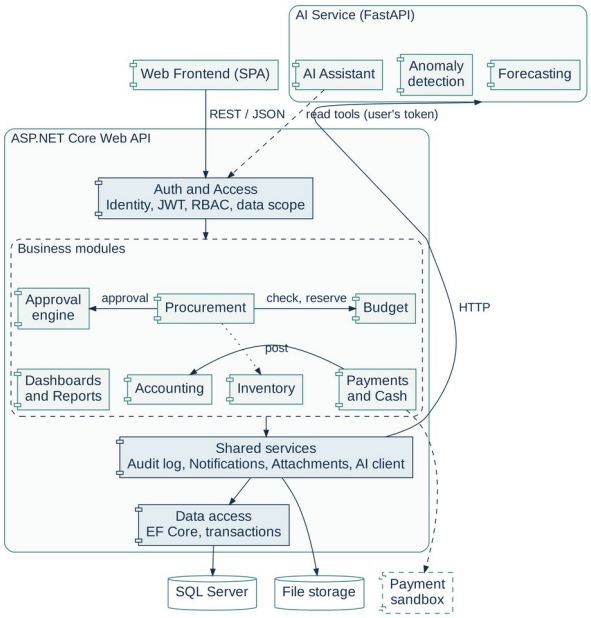


Figure 5.2: Component diagram

6. Additional Deliverables

6.1 API Documentation

The backend exposes a REST API built with ASP.NET Core Web API (modular monolith). The Angular frontend and the AI service use it; nothing else reads the database. This section is the design of that API. The OpenAPI file generated from the code becomes the exact contract, and any change to it goes through a pull request. All values below marked proposed are to be confirmed in the SRD.

6.1.1 General Conventions

Topic	Rule
Base URL and versioning	/api/v1/... The version is in the path; a breaking change creates /api/v2.
Format	JSON (UTF-8), camelCase property names. Dates and times are ISO 8601 in UTC (2026-11-06T09:30:00Z); date-only values are yyyy-MM-dd.
Money and quantities	Decimal numbers, never floating point on the server (amounts 2 decimals, unit prices 4, quantities 3). Every money field has its currencyCode (ISO 4217).
Authentication	Authorization: Bearer <access token> (JWT) on every endpoint except login, refresh and health.
Authorization	Permission policies named Module.Action (for example Budget.Approve) plus data scope (company, branch, department or own data), checked in the backend on every request (FR-AUT-003, FR-AUT-004). A record outside the user’s scope returns 404, so its existence is not revealed.

Topic	Rule
Pagination	GET lists accept page (default 1) and pageSize (default 20, maximum 100). The response is { items, page, pageSize, totalCount }.
Search, filter, sort	search=text, status=Approved, departmentId=3, from=2026-01-01, to=2026-03-31, sort=-createdAt (minus = descending). Unknown parameters are ignored.
State changes	Status changes use action endpoints (POST /purchase-requests/{id}/submit), not a free status field. The service layer allows only the transitions of NUV-S1-03, section 4.2.
Delete	DELETE exists only for records still in Draft. Otherwise the record changes status (Cancelled, Inactive). Financial records are never hard-deleted.
Concurrency	Entities with a RowVersion return rowVersion (Base64). An update must send it back; if the record changed, the API answers 409 CONCURRENCY_CONFLICT and the client reloads (FR-BUD-004).
Idempotency	POST /payments requires the header Idempotency-Key (a GUID created by the client). The same key with the same body returns the first result and creates nothing new; the same key with a different body returns 409 IDEMPOTENCY_KEY_REUSED (FR-PAY-004).
Correlation	The client may send X-Correlation-Id; otherwise the API creates one. It is returned in the response and stored in the audit log and in the application logs.
Errors	RFC 7807 problem details (application/problem+json) with the extra fields code, traceId and, for validation, errors by field.
Audit	Every create, update and status change is written to AuditLogs by the API, never by the client (FR-SYS-002). The API user cannot update or delete audit rows.
Login protection	Login is rate-limited and an account is locked for a short time after repeated failed attempts (proposed: 5 attempts, 15 minutes). Failed logins are audited.

6.1.2 Authentication and Session

Method	Endpoint	Description	Permission
POST	/auth/login	Email and password. Returns accessToken, expiresIn, user and permissions. Sets the refresh token as an httpOnly, Secure, SameSite=Strict cookie.	Anonymous
POST	/auth/refresh	Returns a new access token using the refresh cookie (the refresh token is rotated).	Anonymous (cookie)
POST	/auth/logout	Revokes the refresh token and ends the session.	Authenticated
GET	/auth/me	Current user, company, roles, permissions and data scope. The frontend uses it to build the menu.	Authenticated
POST	/auth/change-password	Changes the password after checking the current one; revokes other sessions.	Authenticated

Proposed token lifetimes: access token 30 minutes, refresh token 7 days. The access token is kept in memory by the frontend, not in localStorage.

6.1.3 Endpoint Catalogue

Paths are relative to /api/v1. List endpoints support the pagination, search and filter rules of 6.1.1. {id} is the numeric id of the record.

Users, roles and organization

Method	Endpoint	Description	Permission
GET	/users, /users/{id}	List and read users (filters: role, department, status).	Users.View
POST	/users	Create a user and set role and department.	Users.Manage
PUT	/users/{id}	Edit a user.	Users.Manage
POST	/users/{id}/activate, /deactivate	Enable or disable an account (no delete).	Users.Manage
PUT	/users/{id}/roles	Replace the roles of a user.	Users.Manage
GET	/roles, /permissions	List roles and the permission catalogue.	Roles.View
POST, PUT	/roles, /roles/{id}	Create or edit a role.	Roles.Manage
PUT	/roles/{id}/permissions	Replace the permissions of a role.	Roles.Manage
GET, PUT	/company	Read or edit the current company (name, tax number, currency).	Company.View / Manage
GET, POST, PUT	/branches, /departments (and /{id})	Manage branches and the department tree.	Organization.View / Manage

Budget and financial planning

Method	Endpoint	Description	Permission
GET, POST, PUT	/financial-periods (and /{id})	Manage financial periods.	Budget.View / Manage
POST	/financial-periods/{id}/close	Close a period; its budgets become read-only.	Period.Close
GET, POST, PUT	/cost-centers, /expense-categories (and /{id})	Manage cost centers and expense categories.	Budget.View / Manage
GET	/budgets, /budgets/{id}	List and read budgets.	Budget.View
POST, PUT	/budgets, /budgets/{id}	Create or edit a budget (Draft only).	Budget.Create
POST	/budgets/{id}/submit, /approve, /reject, /close	Move the budget through its workflow. Approve is refused for the creator.	Budget.Create / Approve
GET, POST	/budgets/{id}/allocations	List or add allocations. Rejected if the total would exceed the budget.	Budget.View / Create
PUT, DELETE	/budgets/{id}/allocations/{allocationId}	Edit or remove an allocation (Draft budget only).	Budget.Create
GET	/budgets/{id}/summary	Planned, Committed, Spent and Available per allocation (FR-BUD-005).	Budget.View
GET, POST	/budget-transfers	List or request a transfer between two allocations.	Budget.View / Transfer
POST	/budget-transfers/{id}/approve, /reject	Decide on a transfer (reason required to reject).	Budget.Approve
GET	/budget-reservations	Read-only list of reservations (created by the system).	Budget.View

Purchase requests and approvals

Method	Endpoint	Description	Permission
GET	/purchase-requests, /{id}	List (scope: own, department or all) and read a request with items.	Request.View
POST, PUT, DELETE	/purchase-requests, /{id}	Create, edit or delete a Draft request.	Request.Create
POST	/purchase-requests/budget-check	Check available budget for the items before submitting (FR-REQ-002).	Request.Create
POST	/purchase-requests/{id}/submit	Submit: checks budget, creates the approval request, notifies approvers.	Request.Create
POST	/purchase-requests/{id}/cancel	Cancel Draft or Submitted; releases any reservation.	Request.Create
GET	/purchase-requests/{id}/timeline	Activity timeline from creation to closure (FR-REQ-007).	Request.View
GET	/approvals/inbox	Items waiting for the current user (FR-REQ-005).	Approval.Decide
GET	/approvals/{id}	Approval request with steps and actions.	Approval.View
POST	/approvals/{id}/steps/{stepId}/ approve, /reject, /return	Decide on a step. A comment is required for reject and return. Refused if the user created the entity.	Approval.Decide
GET, POST, PUT	/approval-rules (and /{id})	Manage approval rules by entity type and amount range.	Approval.Configure

Suppliers, purchase orders and goods receipts

Method	Endpoint	Description	Permission
GET, POST, PUT	/suppliers (and /{id})	Manage suppliers; POST /suppliers/{id}/deactivate for Inactive.	Supplier.View / Manage
GET	/purchase-orders, /{id}	List and read purchase orders with items and received quantities.	PO.View
POST	/purchase-orders	Create from an approved request: purchaseRequestId, supplierId, items.	PO.Manage
PUT	/purchase-orders/{id}	Edit a Draft order.	PO.Manage
POST	/purchase-orders/{id}/issue, /cancel, /close	Issue to the supplier, cancel or close.	PO.Manage
GET	/goods-receipts, /{id}	List and read goods receipts.	Receipt.View
POST, PUT	/goods-receipts, /{id}	Create or edit a Draft receipt against an order (partial delivery allowed).	Receipt.Record
POST	/goods-receipts/{id}/post, /cancel	Post: stock increases for accepted quantity and the order status updates. Cancel after posting reverses stock.	Receipt.Record

Inventory

Method	Endpoint	Description	Permission
GET, POST, PUT	/inventory/items, /inventory/categories (and /{id})	Manage items (SKU, unit, minimum stock) and categories.	Inventory.View / Manage
GET, POST, PUT	/warehouses (and /{id})	Manage warehouses of a branch.	Inventory.View / Manage
GET	/inventory/balances	Quantity on hand by item and warehouse.	Inventory.View
GET	/inventory/movements	Movement history (read-only, append-only table).	Inventory.View
GET	/inventory/low-stock	Items below the minimum level (FR-INV-004).	Inventory.View
POST	/inventory/movements/issue, /transfer, /adjustment	Stock issue (FR-INV-003, Should), transfer between warehouses, adjustment (FR-INV-005, Could).	Inventory.Manage

Supplier invoices, payments and cash

Method	Endpoint	Description	Permission
GET	/supplier-invoices, /{id}	List and read invoices with items.	Invoice.View
POST, PUT	/supplier-invoices, /{id}	Register or edit a Draft invoice linked to an order.	Invoice.Manage
POST	/supplier-invoices/{id}/submit, /approve, /reject, /cancel	Workflow; approve marks the budget reservation as Consumed and posts the accounting entry.	Invoice.Manage / Approve
GET	/supplier-invoices/{id}/match	Three-way match: invoice vs order vs receipt, with differences (FR-PRO-004).	Invoice.View
GET	/payments, /{id}	List and read payments with allocations.	Payment.View
POST	/payments	Create a Pending payment (sandbox or external transfer). Requires Idempotency-Key. Body: supplierId, amount, paymentMethod, paymentReference, allocations.	Payment.Record
POST	/payments/{id}/complete	Confirm the payment. In one transaction: cash transaction, ledger entry, invoice status. Sandbox simulation uses simulate=true.	Payment.Record / Approve
POST	/payments/{id}/fail	Mark as Failed with a reason; the invoice stays unpaid and a retry is allowed.	Payment.Record
POST	/payments/{id}/cancel	Cancel a Pending payment.	Payment.Record
GET, POST, PUT	/cash-accounts (and /{id})	Manage cash and bank accounts.	Cash.View / Manage
GET	/cash-accounts/{id}/transactions	Transactions and running balance of one account.	Cash.View

Accounting, dashboard and reports

Method	Endpoint	Description	Permission
GET, POST, PUT	/accounts (and /{id})	Chart of accounts as a tree.	Ledger.View / Manage

Method	Endpoint	Description	Permission
GET	/ledger-entries, /{id}	Entries with lines (filters: period, account, source).	Ledger.View
POST	/ledger-entries	Create a manual adjusting entry in Draft. System entries are created by the workflows, not by this endpoint.	Ledger.Post
POST	/ledger-entries/{id}/post, /reverse	Post (must balance, period Open) or reverse (creates the opposite entry).	Ledger.Post
GET	/dashboard/summary	Role-based dashboard data: budget cards, pending approvals, recent transactions, stock alerts.	Authenticated
GET	/reports/budget-vs-actual, /reports/expenses, /reports/ledger	Report data with filters by date, department, supplier and status.	Report.View
GET	/reports/{name}/export?format=pdf|xlsx	Export a report.	Report.Export

Notifications, attachments, audit and system

Method	Endpoint	Description	Permission
GET	/notifications	Notifications of the current user (unread filter).	Authenticated
POST	/notifications/{id}/read, /notifications/read-all	Mark as read.	Authenticated
GET, PUT	/me/settings	Language, time zone and notification preferences of the user.	Authenticated
POST	/attachments	Upload a file (multipart): entityType, entityId, file. Type and size are validated.	Attachment.Add
GET	/attachments?entityType=&entityId=, /attachments/{id}/download	List and download. Access follows the permission of the parent record.	Attachment.View
DELETE	/attachments/{id}	Remove while the parent record is in Draft.	Attachment.Add
GET	/audit-logs	Search by user, date, entity type and id; shows old and new values. Read-only.	Audit.View
GET	/health/live, /health/ready	Liveness and readiness (database and AI service).	Anonymous

AI features (the browser calls the .NET API only; the API calls the AI service)

Method	Endpoint	Description	Permission
GET	/ai/forecast/cash-flow?horizon=week|month|quarter	Cash-flow forecast with the baseline, data source and data date (FR-AI-001).	AI.View
GET	/ai/forecast/budget-risk?departmentId=	Probability that a department exceeds its budget (FR-AI-002).	AI.View
GET	/ai/anomalies, /ai/anomalies/{id}	Flagged invoices and transactions with risk score and reason (FR-AI-003).	AI.View
POST	/ai/anomalies/{id}/review	Confirm or dismiss with a comment; the decision is stored.	AI.Review
GET	/ai/recommendations	Stored recommendations (FR-AI-006).	AI.View

Method	Endpoint	Description	Permission
POST	/ai/assistant/query	Ask a question in natural language. The API forwards it with the user's identity; the answer includes data source and date. The query is logged.	AI.Assistant
GET	/ai/assistant/history	Own previous questions.	AI.Assistant

6.1.4 Examples

Create a purchase request

```
POST /api/v1/purchase-requests
Authorization: Bearer <token>

{
  "departmentId": 3, "budgetId": 12, "costCenterId": 5,
  "justification": "Laptops for the new finance team",
  "items": [
    { "itemDescription": "Laptop 15 inch", "quantity": 4, "estimatedUnitPrice": 32000.0000 }
  ]
}

201 Created
{
  "purchaseRequestId": 204, "requestNumber": "PR-2026-0204", "status": "Draft",
  "expectedTotalAmount": 128000.00, "currencyCode": "EGP", "rowVersion": "AAAAAAAAB9E="
}
```

Submit it (budget check fails)

```
POST /api/v1/purchase-requests/204/submit

422 Unprocessable Entity   (application/problem+json)
{
  "type": "https://nuvanta.example/errors/insufficient-budget",
  "title": "Not enough available budget",
  "status": 422, "code": "INSUFFICIENT_BUDGET",
  "detail": "Requested 128000.00 EGP, available 90000.00 EGP in the allocation.",
  "traceId": "00-9f3c...-01"
}
```

Approve a step

```
POST /api/v1/approvals/77/steps/1/approve
{ "comment": "Approved within the department limit" }

200 OK
{ "approvalRequestId": 77, "status": "Approved", "entityType": "PurchaseRequest", "entityId":
204,
  "reservation": { "allocationId": 31, "reservedAmount": 128000.00, "status": "Active" } }
```

Record a payment (idempotent)

```
POST /api/v1/payments
Idempotency-Key: 6b1c1d3e-4c2a-4f1b-9a57-0d2d8e0d7a11

{
  "supplierId": 9, "amount": 45000.00, "paymentMethod": "BankTransfer",
  "paymentReference": "TRF-88231", "paymentDate": "2026-11-06",
  "allocations": [ { "supplierInvoiceId": 118, "allocatedAmount": 45000.00 } ]
}

201 Created   { "paymentId": 56, "status": "Pending", "rowVersion": "AAAAAAAAB9I=" }
(sending the same request again returns the same payment, status 200, and creates nothing)
```

6.2 Testing & Validation

The testing plan shows how the team proves that NUVANTA meets its requirements and KPIs. The financial and permission rules get the most attention, because they are the main risks of the project (R2 to R5, R8). All tests run automatically in the CI pipeline on every pull request.

6.2.1 Test Levels and Tools

Level	What it checks	Tools
Unit (backend)	Business rules in the service layer: budget availability, status transitions, approval routing, ledger balance, payment allocation, three-way match.	xUnit, FluentAssertions, NSubstitute; coverage with Coverlet
Unit and component (Angular)	Services, guards, interceptors, pipes and components of the frontend.	Angular testing utilities with Vitest (the test runner configured in the project)
Integration (database)	Real SQL Server: constraints, unique and filtered indexes, triggers, RowVersion, transactions and rollback, views.	WebApplicationFactory, Testcontainers for SQL Server, Respawn to reset data
API tests	Every endpoint: status codes, validation, error codes, pagination, idempotency, response shape against the OpenAPI file.	xUnit with WebApplicationFactory; Postman or Bruno collection run in CI
Security and permission	Each role against each endpoint, data scope, Maker–Checker, token handling, sensitive fields.	Data-driven integration tests generated from the role permission matrix
End-to-end (UI)	The demo flow in a real browser: login, budget, request, approval, receipt, invoice, payment, dashboard.	Playwright
Performance	Response time of the main endpoints and the dashboard under expected load.	k6
AI service	Forecast error against a baseline, anomaly precision and recall, permission-respecting assistant.	pytest, scikit-learn metrics
Usability and acceptance	Real users complete the main tasks.	Test script and questionnaire (6.2.4)

6.2.2 Workflow and Maker–Checker Test Cases

ID	Scenario	Expected result	Req.
TC-WF-01	The creator of a purchase request tries to approve it.	403 MAKER_CHECKER_VIOLATION; status unchanged.	FR-AUT-006
TC-WF-02	A request of 5,000 EGP and one of 250,000 EGP are submitted.	Each is routed to the approvers defined by the matching approval rule.	FR-REQ-003
TC-WF-03	An approver rejects or returns without a comment.	400 VALIDATION_ERROR; comment is mandatory.	FR-REQ-004
TC-WF-04	An approver returns a request.	Requester is notified; request goes back to editing; budget is not reserved.	FR-REQ-004
TC-WF-05	A user opens the approval inbox.	Only items assigned to the user or to their role are listed.	FR-REQ-005
TC-WF-06	Try illegal transitions (approve a Draft request, edit a Paid invoice, complete a Cancelled payment).	409 INVALID_STATE_TRANSITION for each.	NUV-S1-03 4.2
TC-WF-07	Submit a request without items.	Rejected: at least one item is required.	FR-REQ-001
TC-WF-08	Approve a request and then follow its timeline.	Every step with user, time and decision is listed.	FR-REQ-007

6.2.3 Financial Integrity Test Cases

ID	Scenario	Expected result	Req.
TC-FIN-01	Allocations are added so that their sum exceeds the budget total.	Rejected; nothing saved.	FR-BUD-002
TC-FIN-02	Create an allocation or transfer with a negative or zero amount.	400 VALIDATION_ERROR; nothing saved.	FR-BUD-003
TC-FIN-03	Two requests reserve the same allocation at the same time and only one fits.	One succeeds, the other gets INSUFFICIENT_BUDGET; available never goes below zero.	FR-BUD-004
TC-FIN-04	Approve a request, then cancel another approved one.	Reservation is Active after approval and Released after cancel; Committed changes accordingly.	FR-REQ-006
TC-FIN-05	After a fixed sequence of events, read the budget summary.	Planned, Committed, Spent and Available equal the hand-calculated values.	FR-BUD-005
TC-FIN-06	Post a ledger entry whose debit and credit totals differ.	422 LEDGER_UNBALANCED; the database trigger also blocks it.	FR-ACC-002
TC-FIN-07	Update or delete a Posted entry or its lines.	Refused by the service and by the database; correction needs a reversing entry.	FR-ACC-003
TC-FIN-08	Force a failure while completing a payment (after the cash step, before the ledger step).	Whole transaction rolls back: cash, budget, ledger and invoice are unchanged.	FR-PAY-006
TC-FIN-09	Send the same payment request twice with one Idempotency-Key, also in parallel.	One payment exists; the second call returns the first result.	FR-PAY-004
TC-FIN-10	Record two payments with the same PaymentReference.	The second is rejected (unique constraint).	FR-PAY-004

ID	Scenario	Expected result	Req.
TC-FIN-11	A payment fails.	Status Failed with a reason; invoice stays unpaid; a retry is possible.	FR-PAY-005
TC-FIN-12	Allocate more than the payment amount or more than the invoice total.	422 PAYMENT_ALLOCATION_INVALID.	NUV-S1-03 5.5
TC-FIN-13	Post a goods receipt with accepted and rejected quantities.	Stock increases by the accepted quantity only; balance equals the sum of movements.	FR-INV-002
TC-FIN-14	Receive more than the ordered quantity.	422 QUANTITY_EXCEEDED.	FR-PRO-003
TC-FIN-15	Register an invoice whose price or quantity differs from the order or receipt.	The difference is flagged and the invoice is held for review.	FR-PRO-004
TC-FIN-16	Post an entry into a Closed financial period.	422 PERIOD_CLOSED.	NUV-S1-03 7
TC-FIN-17	Run the reconciliation after the full demo flow.	Budget, cash, ledger and stock agree; total debit equals total credit.	NFR-INT-01

6.2.4 Usability and Acceptance Testing

- Five to eight classmates or contacts, each with the role of an Employee, a Manager or an Accountant, follow a written script on the demo data.
- Script (the end-to-end demo): create a budget, submit a purchase request, approve it, record goods receipt, register the invoice, record a payment, then check the dashboard and the audit log.
- Measured: task success rate (target at least 90%), time per task, errors, and a questionnaire (target at least 4 of 5).
- Findings are logged in Jira, ranked by severity, and the critical ones are fixed before the final delivery (risk R13).

6.2.5 Defects and Exit Criteria

Severity	Meaning	Rule
Critical	Wrong financial figure, duplicate payment, permission leak, data loss, or the demo flow cannot be completed.	Fixed before delivery; zero open at the end.
High	A Must requirement fails but a work-around exists.	Fixed before delivery.
Medium	A Should requirement fails or the screen is confusing.	Fixed if time allows; otherwise documented.
Low	Cosmetic issue.	Backlog.

Exit criteria for the final delivery: no open Critical bug; pass rate at least 95%; coverage at least 70% on financial and permission logic; at least 90% of requirements covered by tests; the end-to-end demo script passes on the demo environment; all permission tests pass.

6.2.6 Test Execution in CI

- Unit, integration, API and permission tests run on every pull request; a pull request cannot be merged while they fail (100% of pull requests are reviewed).
- E2E smoke tests run after each deployment to staging; the full E2E and performance runs are executed before each milestone and before the final demo.
- Coverage and test reports are stored as pipeline artifacts.

6.3 Deployment Strategy

This section explains how NUVANTA is built, released and run. It complements the deployment diagram (5.2) and the component diagram (5.3). The hosting platform is not decided yet; the proposal is one Linux server running the containers, to be confirmed by the team.

6.3.1 Environments

Environment	Purpose	How it runs
Local development	Each developer works on a feature branch.	Docker Compose starts SQL Server, the API and the AI service; the Angular app runs with ng serve and a proxy to the API. Demo seed is loaded.
CI	Automatic build and tests on every pull request.	Short-lived containers on the pipeline runner; empty database created for each run.
Staging	Rehearsal of the release with demo data.	Same containers and configuration as the demo environment, with its own database.
Demo / production	The environment shown to the instructor and evaluators.	Containers behind a reverse proxy with HTTPS and scheduled backups.

6.3.2 CI/CD Pipeline

The pipeline is built with GitHub Actions on the shared repository. Work follows the planned way of working: feature branch, pull request with at least one reviewer, then merge into main.

#	Stage	What happens	Fails when
1	Build	dotnet build, ng build, lint for C# and TypeScript, install of the Python dependencies.	Compile or lint error
2	Unit tests	xUnit, Vitest and pytest with coverage.	A test fails or coverage of financial and permission logic is below 70%
3	Integration and API tests	SQL Server container, migrations, integration tests, API and permission tests.	A test fails
4	Security scan	Vulnerable packages (dotnet and npm), CodeQL, secret scan.	A high-severity finding
5	Docker images	Build the web, api and ai images, tag with the commit and version, push to the registry.	Build error
6	Deploy to staging	Apply the migration script, then pull the new images and restart the containers.	Migration or start-up error
7	Smoke and E2E	Health endpoints and the Playwright smoke flow on staging.	A check fails
8	Manual approval	A team member reviews staging and approves the release.	Not approved
9	Deploy to demo / production	Same steps as staging with the production configuration; the previous image tag is kept for rollback.	Health check fails

Stages 1 to 5 run on every pull request; stages 6 to 9 run when main changes, and release tags (v1.0.0) mark the delivered versions.

6.3.3 Configuration and Secrets

- Settings come from appsettings files per environment and from environment variables; the environment variable always wins.
- Secrets (database password, JWT signing key, AI service key, provider keys) are never committed. They are stored in the pipeline secret store and in a .env file on the server that is excluded from Git.
- The Angular environment files contain only public values such as the API address.
- HTTPS is mandatory in staging and production; the API allows only the frontend origin (CORS) and sends security headers.

6.3.4 Database Deployment

- EF Core migrations are committed with the code. The pipeline creates an idempotent SQL script and applies it before the new API version starts; the API never migrates the production database by itself.
- Destructive changes follow expand and contract: add the new structure, move the code, remove the old structure in a later release.
- The reference seed runs in every environment; the demo seed only in development, staging and demo.
- The application database account has the minimum rights: no schema changes, and insert and read only on the append-only tables (AuditLogs, StockMovements, ApprovalActions).

6.3.5 Release and Rollback

- Each release has a version tag and matching image tags.
- Rollback: redeploy the previous image tag. If the release included a migration, the migration is written so the previous version still works with it; otherwise the database is restored from the pre-release backup.
- A backup is taken automatically before every production migration.

6.3.6 Delivery Checklist

☐ Demo environment is running with the final images and demo data
☐ HTTPS certificate valid and all secrets set
☐ Backups configured and a restore tested
☐ Health checks green and the end-to-end demo script passed on this environment
☐ Accounts for every role prepared and tested
☐ Repository, documentation and OpenAPI file up to date and tagged